

1 Highlights

2 **GPU-Accelerated DMRG on AMD MI300X:** 3 **Systematic Benchmarking and Lessons from Failed Optimizations**

4 Yaakoub El Khamra

- 5 • Systematic benchmark: 11 DMRG implementations, 3 physics models,
6 $L=8-128$, $\chi=20-256$.
- 7 • At $\chi \leq 50$, a single CPU core outperforms the 304-CU MI300X in 93%
8 of the tested configurations (single-thread harness; see § 4.1).
- 9 • CPU LAPACK SVD consumes 97–98% of per-sweep time at $\chi=256$,
10 capping GPU speedups.
- 11 • Three prototyped algorithmic variants (Block-Davidson with host-side
12 SVD; cross-segment batched GEMM; Chebyshev-filtered subspace iter-
13 ation) achieve no measured win at $\chi \leq 256$; analytical work-multipliers
14 ($\alpha \gtrsim 3-4$) predict the failure ahead of measurement (§5.4). Newton–
15 Schulz is discussed as a CPU prototype only.
- 16 • Randomized SVD wins $5-6\times$ only in 2-site DMRG; HIP graph capture
17 wins $1.2-1.8\times$ in parallel DMRG.
- 18 • Multi-GPU segment-decomposition (4-MI300X `pdmrg-multi-gpu`) and
19 Riemannian-manifold polish optimizers (`rlbfgs-gpu`, `radam-gpu`) are
20 reported as exploratory directions consistent with their analytical bounds;
21 full $N=10$ campaigns in their predicted regimes of competitiveness are
22 deferred (§5.7).

GPU-Accelerated DMRG on AMD MI300X: Systematic Benchmarking and Lessons from Failed Optimizations

Yaakoub El Khamra^{a,*}

^a, ,

Abstract

We present a systematic performance study of eleven density matrix renormalization group (DMRG) implementations, ranging from Python/CPU to hand-tuned GPU kernels in HIP/C++ targeting the AMD Instinct MI300X. The implementations span single-site and two-site DMRG, real-space parallel DMRG with domain decomposition, and an additive two-level parallel variant (A2DMRG), tested across three physics models: the Heisenberg spin- $\frac{1}{2}$ chain ($d = 2$, real), a Josephson junction array ($d = 5$, complex), and the transverse-field Ising model at criticality ($d = 2$, real). Benchmarks cover system sizes $L = 8$ –128 and bond dimensions $\chi = 20$ –256 on a single MI300X GPU with 192 GB HBM3.

The GPU implementations achieve speedups over optimized CPU code (quimb with OpenBLAS) that depend on both bond dimension and system size: at $\chi = 128$ the GPU wins 100% of short-chain ($L \leq 20$) configurations but only 50% at $L \geq 100$, while at $\chi \leq 50$ the CPU wins all medium-to-large systems ($L \geq 32$). In 27 of 29 CPU-win configurations (93%), a *single* CPU core already outperforms the 304-compute-unit MI300X—the GPU loses not to parallelized CPU BLAS, but to kernel launch latency on small matrices. (Note: the CPU harness is hardcoded to one thread; the 93% reflects the single-thread setting, not a multi-thread sweep; see § 4.1.) Two-site DMRG converges $3.3\times$ faster than single-site at large system sizes ($L = 100$, $\chi = 128$: 53 s vs. 177 s).

We report extensive negative results on algorithmic optimizations designed to exploit the MI300X’s BLAS-3 throughput. Three independent algorithmic substitutions were prototyped on top of the GPU baselines: a Block-Davidson eigensolver ($b=4$) with host-side LAPACK SVD, a cross-

*Corresponding author.

segment batched GEMM dispatch in parallel DMRG, and a Chebyshev-filtered subspace iteration in place of single-vector Lanczos. Each admits a closed-form analytical work-multiplier ($\alpha \gtrsim 3\text{--}4$ at $\chi \leq 256$) that predicts a non-competitive result before measurement; the experiments confirm 1.4–5.7 $\times$ slowdowns where benchmarked at the standard statistical level, and the Chebyshev variant was not benchmarked at $N=10$ in light of its analytical bound (§5.4). Newton–Schulz polar decomposition is discussed as a CPU-prototype alternative in §3 but was not measured on GPU in this study. Cross-segment batched GEMM dispatch is slower in 18 of 19 configurations tested. Profiling reveals the root cause: CPU LAPACK SVD consumes 97–98% of per-sweep wall time at $\chi = 256$, and iterative GPU replacements cannot match LAPACK’s optimized QR-iteration algorithm at these matrix sizes. Two further GPU directions — 4-device multi-GPU segment-decomposition and Riemannian-manifold polish optimizers — are reported as exploratory: analytical bounds and preliminary measurements identify their regimes of potential competitiveness, full statistical campaigns deferred. All source code, benchmark scripts, raw data, and the JSON-driven campaign runner that produces fully self-describing benchmark output are publicly available.

Keywords: DMRG, GPU computing, tensor networks, AMD MI300X, performance benchmarking, matrix product states

PROGRAM SUMMARY

Program Title: `dmrg-implementations`
CPC Library link to program files: (to be added on acceptance)
Developer’s repository link:
<https://github.com/yye00/dmrg-implementations>
Code Ocean capsule: (not used)
Licensing provisions: MIT
Programming language: C++17/HIP (~32 kLOC), Python 3 (~22 kLOC, excluding tests and virtual environments), MPI (mpi4py) for A2DMRG; GPU code in HIP/C++ targeting ROCm 7.2 (LOC counted via `find gpu-rocm cpu -name '*.cpp' -o -name '*.h' | grep -v build | xargs wc -l`)
External libraries/dependencies: rocBLAS, rocsolver, HIP (ROCm 7.2+); OpenBLAS (version and linkage captured per-run via `ldd`; see provenance

88 note in § 6.5), LAPACK; quimb 1.8, NumPy; mpi4py for parallel Python
89 (cotengra is listed as a dependency but is not invoked by the runner; see
90 § 6.5)
91 *Supplementary material:* none

92 *Nature of problem:* Ground-state computation of one-dimensional quantum
93 lattice models via the density matrix renormalization group (DMRG).
94 Per-bond work is dominated by tensor contractions (GEMM, $O(\chi^3 d^2)$) and
95 singular value decompositions, with moderate bond dimensions
96 $\chi \in [20, 256]$ straddling the CPU–GPU crossover.

97 *Solution method:* Eleven DMRG variants spanning single-site and two-site
98 sweeps, domain-decomposition parallel DMRG [13], an additive two-level
99 variant (A2DMRG) [19], and three algorithmic alternatives designed to
100 increase BLAS-3 utilization (Block-Davidson eigensolver (with host-side
101 LAPACK SVD)). All GPU variants share a common env-var-gated ablation
102 framework (six micro-optimizations: DEVICE_K, LANCZOS_GRAPH,
103 RSVD, SPARSE_MPO, FUSE_LANCZOS, D_PAD) so each flag’s effect
104 can be measured in isolation on the same binary. Benchmarks run across
105 three physics models (Heisenberg chain, Josephson-junction array,
106 transverse-field Ising) at system sizes up to $L = 128$.

107 *Additional comments:* All results in this paper were generated on the AMD
108 Instinct MI300X (gfx942, ROCm 7.2). Each result JSON embeds git commit
109 SHA, binary SHA-256, runtime CLI, and environment variable snapshot for
110 reproducibility.

111 1. Introduction

112 The density matrix renormalization group (DMRG) [1, 2] is the method
113 of choice for computing ground states of one-dimensional quantum many-
114 body systems. In the modern tensor network formulation [3, 4], DMRG
115 variationally optimizes a matrix product state (MPS) by sweeping through
116 the chain and solving local eigenvalue problems at each bond. The compu-
117 tational cost is dominated by tensor contractions (matrix–matrix multipli-
118 cations) and singular value decompositions (SVDs), both of which scale as
119 $O(\chi^3 d^2)$ per bond, where χ is the bond dimension and d is the local Hilbert
120 space dimension.

121 Graphics processing units (GPUs) are natural candidates for accelerating
122 DMRG, since the tensor contractions map directly to general matrix–matrix

123 multiplication (GEMM), the workload for which modern GPUs achieve near-
 124 peak throughput. The AMD Instinct MI300X, with 304 compute units,
 125 192 GB of HBM3 memory providing 5.3 TB/s bandwidth, and matrix fused
 126 multiply-add (MFMA) units delivering 81.7 TFLOPS of FP64 matrix through-
 127 put, represents a compelling target for the GEMM-dominated inner loop.
 128 The field has mature open-source DMRG libraries: ITensor [5], TeNPy [?
 129], and Block2 [?] represent the production CPU and hybrid CPU/GPU
 130 ecosystem. Prior work has also explored GPU acceleration targeting NVIDIA
 131 hardware [7?], but a systematic study of the MI300X—and of when GPU
 132 acceleration helps or does not—has been lacking.

133 Two questions are open. First, at what problem sizes does GPU acceler-
 134 ation become worthwhile? The practical performance depends on a complex
 135 interplay of bond dimension χ (which determines GEMM sizes), local dimen-
 136 sion d (which determines batch counts), kernel launch overhead, and the SVD
 137 bottleneck, making the answer non-obvious a priori. Second, can algorithmic
 138 modifications designed to increase GEMM utilization extend the favorable
 139 regime to smaller bond dimensions?

140 We address both questions through a systematic benchmarking study
 141 spanning multiple CPU and GPU DMRG implementations across three physics
 142 models at system sizes $L = 8$ –128 and bond dimensions $\chi = 20$ –256. We
 143 find a narrow window of GPU advantage at large χ and small L , and show
 144 through systematic negative results that this window cannot be widened
 145 at moderate bond dimensions by several algorithmic modifications designed
 146 to exploit BLAS-3 throughput—all of which failed. Remarkably, in 27 of
 147 29 configurations where the CPU wins, a *single* CPU core already outper-
 148 forms the 304-compute-unit MI300X; the GPU loses not to parallelized CPU
 149 BLAS, but to kernel launch latency on matrices small enough to fit in L2
 150 cache. These negative results delineate a boundary between regimes where
 151 massive GPU parallelism adds value and regimes where a single CPU core’s
 152 low-latency sequential execution cannot be beaten.

153 The paper is organized as follows. Section 2 describes the DMRG al-
 154 gorithm and its GPU implementation, with emphasis on the operations
 155 that motivate GPU porting. Section 3 details the algorithmic variants,
 156 framing each as a hypothesis about how to shift computation toward the
 157 GPU’s strengths. Section 4 specifies the hardware, software, and benchmark
 158 methodology. Section 5 presents the performance results. Section 6 analyzes
 159 the failures and identifies the SVD bottleneck as the root cause. Section 7
 160 summarizes our findings.

161 2. Methods

162 This section describes the DMRG algorithm and its GPU implementa-
 163 tion. The formalism is standard [3], but we present it with attention to
 164 the computational primitives at each step, since the cost structure of these
 165 primitives—which operations are BLAS-3, which are BLAS-2, and which are
 166 factorizations—determines where GPU acceleration can and cannot help.

167 2.1. Tensor trains and matrix product states

168 We consider a one-dimensional quantum lattice of L sites with open
 169 boundary conditions. The Hilbert space has dimension d^L , where d is the lo-
 170 cal basis size (e.g., $d = 2$ for spin- $\frac{1}{2}$), making exact diagonalization intractable
 171 beyond $L \approx 20$. The key insight enabling DMRG is the *area law* of entangle-
 172 ment [8]: ground states of gapped local Hamiltonians in one dimension have
 173 entanglement entropy bounded by a constant independent of system size,
 174 which means they can be efficiently represented by matrix product states
 175 (MPS) with moderate bond dimension χ .

176 An MPS (equivalently, a tensor train in the numerical mathematics lit-
 177 erature [9]) parametrizes the d^L -dimensional state vector as a product of L
 178 three-index tensors:

$$|\psi\rangle = \sum_{\sigma_1, \dots, \sigma_L} A_1^{\sigma_1} A_2^{\sigma_2} \cdots A_L^{\sigma_L} |\sigma_1 \cdots \sigma_L\rangle, \quad (1)$$

179 where each $A_i^{\sigma_i}$ is a $\chi_{i-1} \times \chi_i$ matrix (with $\chi_0 = \chi_L = 1$) and $\sigma_i = 1, \dots, d$
 180 labels the local basis states. The bond dimension $\chi = \max_i \chi_i$ controls the
 181 expressiveness of the ansatz: $\chi = 1$ yields a product state, while $\chi = d^{L/2}$
 182 can represent any state exactly. For ground states of local Hamiltonians,
 183 $\chi = O(\text{poly}(L))$ typically suffices for fixed accuracy, giving an exponential
 184 compression from d^L to $O(L\chi^2 d)$ parameters.

185 2.2. Canonical forms

186 Maintaining the MPS in canonical form is essential for numerical stability
 187 and for the local eigenvalue problem that drives DMRG. It also introduces
 188 the first bottleneck relevant to GPU porting: the SVD-based truncation that
 189 maintains canonical form will prove to be the dominant computational cost
 190 (Section 5.6).

191 An MPS is in *left-canonical form* up to site k if each tensor A_i ($i < k$)
 192 satisfies $\sum_{\sigma} A_i^{\sigma\dagger} A_i^{\sigma} = I$, i.e., each tensor is a (partial) isometry when viewed

193 as a matrix from left to right. Similarly, *right-canonical form* from site k
 194 means $\sum_{\sigma} A_i^{\sigma} A_i^{\sigma\dagger} = I$ for $i > k$. An MPS in *mixed-canonical form* centered
 195 at site k is left-canonical for $i < k$ and right-canonical for $i > k$, so that the
 196 tensor at site k (the “active” tensor) contains all the variational freedom.

197 Canonical form is established and maintained by QR or SVD decompo-
 198 sitions. To left-canonicalize site i , reshape A_i into a $(\chi_{i-1}d) \times \chi_i$ matrix and
 199 compute the thin QR factorization $A_i = QR$; replace $A_i \leftarrow Q$ and absorb
 200 R into A_{i+1} . To truncate the bond dimension (as needed after two-site op-
 201 timization), SVD is used instead: $A_i = USV^{\dagger}$, truncate to the $\chi_{\max}$ largest
 202 singular values, and split the factors between sites i and $i + 1$. The trun-
 203 cation error $\epsilon = \sqrt{\sum_{k > \chi_{\max}} s_k^2}$ is bounded by the discarded singular values,
 204 providing a controlled approximation.

205 2.3. Matrix product operators

206 The Hamiltonian H is represented as a matrix product operator (MPO)
 207 [3, 4]:

$$H = \sum_{\sigma, \sigma'} W_1^{\sigma_1 \sigma'_1} W_2^{\sigma_2 \sigma'_2} \dots W_L^{\sigma_L \sigma'_L} |\sigma\rangle \langle \sigma'|, \quad (2)$$

208 where each $W_i^{\sigma_i \sigma'_i}$ is a $D_{i-1} \times D_i$ matrix and $D = \max_i D_i$ is the MPO
 209 bond dimension. For nearest-neighbor Hamiltonians, D is a small constant:
 210 $D = 5$ for the Heisenberg model, $D = 3$ for the transverse-field Ising model,
 211 and $D = 4$ for the Josephson junction array. The MPO provides an exact,
 212 compact representation of the Hamiltonian that enables efficient computation
 213 of expectation values and H_{eff} applications in $O(\chi^2 d D)$ operations per bond.

214 2.4. The DMRG sweep

215 The density matrix renormalization group (DMRG) [1, 2] variationally
 216 optimizes the MPS by sweeping through the chain. In the modern formula-
 217 tion [3], a left-to-right sweep followed by a right-to-left sweep constitutes one
 218 “sweep” of the algorithm. At each bond b , the algorithm:

- 219 1. Forms the *effective Hamiltonian* H_{eff} by contracting the left environ-
 220 nment L_{env} (encoding all sites $< b$), the MPO tensor(s) W_b , and the
 221 right environment R_{env} (encoding all sites $> b$) into a linear operator
 222 acting on the active tensor(s) at bond b .
- 223 2. Solves the *local eigenvalue problem*

$$H_{\text{eff}}|\theta\rangle = E|\theta\rangle \quad (3)$$

224 for the ground-state energy E and optimal local tensor θ , using a Krylov
 225 eigensolver (Lanczos [25]).
 226 3. *Restores canonical form*: decomposes the updated tensor via SVD or
 227 QR and shifts the canonical center to the next bond.
 228 4. *Updates the environment*: incorporates the newly optimized tensor into
 229 L_{env} (left sweep) or R_{env} (right sweep) for the next bond.

230 The environment tensors L_{env} and R_{env} are rank-3 tensors of dimension
 231 $\chi \times D \times \chi$ that encode the contraction of all MPS and MPO tensors to the
 232 left (or right) of the active bond. They are updated incrementally as the
 233 sweep progresses, at a cost of $O(\chi^2 d D + \chi^2 D^2)$ per bond. The total cost per
 234 sweep is $O(L \chi^3 d^2 D)$ for the eigensolve (dominated by the H_{eff} matrix–vector
 235 product) plus $O(L \chi^3 d)$ for the SVD/QR step.

236 Of these operations, steps 1 and 4 are tensor contractions that map to
 237 GEMM—precisely the workload where the MI300X’s 81.7 TFLOPS matrix
 238 throughput should dominate. Step 2 is a sparse Krylov iteration (BLAS-1/2
 239 dominated), and step 3 is a dense factorization. The GPU porting strategy
 240 is therefore to maximize the fraction of time spent in steps 1 and 4. Whether
 241 this fraction is large enough to overcome launch overhead at moderate χ is
 242 the central empirical question of this paper; Section 5 reports the answer.

243 In *single-site* DMRG, one MPS tensor is optimized at a time, giving
 244 an eigensolve on a vector of dimension χd . The bond dimension is fixed
 245 throughout the sweep, which is efficient but can trap the optimization in
 246 local minima if the initial bond dimension profile is poor.

247 The single-site implementation in this work is the *strict* variant: local
 248 eigensolve followed by exact SVD truncation, with no density-matrix per-
 249 turbation. The Hubig–McCulloch–Schollwöck–Wolf DMRG3S extension [?
 250] restores the ability to grow Schmidt sectors at fixed bond dimension via
 251 a noise term $\alpha \sum_a (H_a |\psi\rangle)(H_a |\psi\rangle)^\dagger$; we do not implement it here. Conse-
 252 quently, our single-site results should be read as a lower bound on what an
 253 expansion-augmented single-site DMRG would achieve, and the convergence-
 254 sweep counts reported in Section 5.2 reflect this limitation.

255 2.5. Two-site DMRG

256 In two-site DMRG [10], pairs of adjacent MPS tensors are optimized
 257 simultaneously. The two-site tensor θ has dimensions $\chi_L d \times d \chi_R$, and after
 258 optimization it is split by SVD:

$$\theta = U S V^\dagger, \quad (4)$$

where S is truncated to the $\chi_{\max}$ largest singular values. This allows the bond dimension to *adapt* during the sweep, which is critical for convergence from a random initial state: new Schmidt coefficients can appear that were absent in the initial MPS. The cost per bond is $O(\chi^2 d^2 D)$ for the eigensolve and $O(\chi^3 d^2)$ for the SVD, both scaling with d^2 rather than d compared to single-site.

We precompute fused MPO tensors $\widetilde{W}[b] = W_L[b] \otimes W_R[b]$ for each bond b , so the two-site H_{eff} application uses the same three-step GEMM pattern as single-site but with $d \rightarrow d^2$. The larger effective dimension increases the GEMM sizes, which should favor the GPU—but also increases the SVD cost quadratically, which may negate the advantage. Section 5.2 quantifies this tradeoff.

2.6. GPU implementation

The GPU implementation is where the promise of GPU acceleration is realized—or not. The design goal was to maximize the BLAS-3 fraction of the inner loop: every tensor contraction is a GEMM, and the only non-GEMM operations are the Lanczos iteration’s vector arithmetic and the SVD factorization.

All GPU implementations target the AMD MI300X via HIP/C++ and use rocBLAS [11] for BLAS operations. The code is templated on the scalar type, supporting both `double` (real) and `hipDoubleComplex` (complex) through a `ScalarTraits` dispatch layer.

The application of H_{eff} to a vector, the innermost operation of the Lanczos eigensolver, is implemented as three batched GEMM steps:

1. **Step 1:** $D \times d^2$ batched GEMMs contracting L_{env} slices with θ slices,
2. **Step 2:** One dense GEMM contracting the intermediate result with the fused MPO $\widetilde{W}$,
3. **Step 3:** $d^2 \times D$ batched GEMMs accumulating the contraction with R_{env} slices into the output.

The pointer arrays for batched GEMMs are computed by lightweight GPU kernels (typically < 64 threads), avoiding host-to-device DMA that would otherwise introduce pipeline stalls [12].

The Lanczos eigensolver runs entirely on the GPU using device-pointer mode for all rocBLAS reductions (`dot`, `nrm2`), eliminating implicit stream synchronizations. Only the final tridiagonal eigensolve (a $k \times k$ problem with

294 $k \approx 15\text{--}50$) is performed on the CPU via LAPACK `dstev`. SVD path dif-
 295 fers by variant family. The primary `-gpu` variants (`dmrg-gpu`, `dmrg2-gpu`,
 296 `pdmrg-gpu`) perform SVD on the GPU via `rocsolver_gesvd_auto`, with an
 297 optional randomized-SVD path (`opts_.rsvd`); the interior SVD for `pdmrg-gpu`
 298 also uses GPU `rocsolver`, while the segment-boundary merge uses a CPU
 299 `accurate_svd` for numerical stability. The `-opt` variants are heterogeneous
 300 in their SVD path: `dmrg-gpu-opt` and `dmrg2-gpu-opt` are the bundle of
 301 Block-Davidson eigensolver + host-side LAPACK SVD (`dgesvd` / `zgesvd`,
 302 QR-iteration based) discussed in Section 5.4; `pdmrg-gpu-opt` defaults to
 303 GPU `rocsolver` SVD like its baseline counterpart and exposes a `-cpu-svd`
 304 flag for opt-in host-side SVD (not benchmarked at the headline statistical
 305 level in this paper). The host-side SVD bundle is motivated by profiling
 306 which showed GPU `rocsolver` provides only a 13% improvement over CPU
 307 LAPACK at $\chi = 256$ and is slower at $\chi \leq 128$ (see Section 5.6). The 97–98%
 308 CPU-SVD figure in Section 5.6 and Table 11 pertains to a `-opt` variant with
 309 the host-side path (the precise variant is unverified; see table caption); the
 310 primary `-gpu` variants and `pdmrg-gpu-opt` use GPU `rocsolver` by default.

311 This three-step GEMM structure is the operation we hoped would be
 312 the GPU’s strength. At $\chi = 128$ with $d = 2$, the individual GEMMs are
 313 128×128 —large enough to occupy the MI300X’s compute units. At $\chi = 50$,
 314 they shrink to 50×50 , completing in microseconds while each kernel launch
 315 costs 5–10 μs . Section 5.1 maps the resulting crossover surface in (L, χ)
 316 space.

317 2.7. Parallel DMRG

318 Serial GPU DMRG is limited by the crossover boundary described above:
 319 below a certain χ , the per-bond GPU overhead exceeds the CPU’s. Parallel
 320 DMRG was motivated by the hope that segment parallelism could amortize
 321 this per-bond overhead across multiple independent sweeps, extending the
 322 GPU-favorable regime to smaller bond dimensions.

323 Our parallel DMRG implementation follows the real-space decomposition
 324 of Stoudenmire and White [13]. The L -site chain is partitioned into P seg-
 325 ments, each assigned to an independent HIP stream and rocBLAS handle.
 326 The algorithm proceeds in three phases:

- 327 1. **Warmup:** Full-chain *single-site* DMRG sweeps on a single stream to
 328 establish an initial converged MPS at fixed bond dimension χ . Single-
 329 site is used because the eigensolver problem is smaller (χd vs. χd^2 for
 330 two-site), making warmup cheaper.

- 331 2. **Outer loop:** Parallel two-site segment sweeps (one `std::thread` per
332 segment, each on its own HIP stream) using staggered directions, fol-
333 lowed by boundary merge-and-optimize coupling via the Stoudenmire
334 $V = \Lambda^{-1}$ prescription.
- 335 3. **Polish:** Full-chain *single-site* sweeps until convergence. Single-site is
336 again used for the polish phase, as the bond dimension is already es-
337 tablished and single-site sweeps are cheaper per bond.

338 The coupling step is essential: segment sweeps disrupt the global en-
339 tanglement structure of the MPS, and only boundary coupling or full-chain
340 resweeping restores the correct ground state. Experiments with boundary-
341 only coupling sweeps (optimizing only sites near segment boundaries) failed
342 to converge (energy errors > 8), confirming that coupling at every boundary
343 is necessary. Section 5.3 reports that this serial coupling requirement limits
344 parallel DMRG to being slower than serial two-site DMRG at all tested sizes
345 on a single GPU.

346 3. Algorithmic variants

347 The baseline GPU DMRG of Section 2 wins at high χ and small L
348 but loses at moderate bond dimensions, where kernel launch overhead dom-
349 inates the small GEMMs and CPU SVD consumes 97–98% of wall time
350 (Section 5.6). This section describes five algorithmic modifications, each mo-
351 tivated by a specific hypothesis about how to shift computation toward the
352 GPU’s strengths. All five hypotheses are addressed in Section 5.4: each was
353 either ruled out by an analytical work-multiplier bound at the $\chi \leq 256$ scale
354 of interest or measured to be non-competitive (or both); none improved over
355 the baselines.

356 3.1. Newton–Schulz polar decomposition (CPU prototype; not deployed on 357 GPU)

358 *Hypothesis:* Replacing the BLAS-2-dominated SVD with GEMM itera-
359 tions should shift the dominant computation to the GPU’s strength, reducing
360 or eliminating the CPU SVD bottleneck.

361 The Newton–Schulz iteration computes the unitary polar factor U of a
362 matrix $A = UP$ (where P is positive semidefinite) via the recurrence [14, 15]

$$X_0 = A/\|A\|_F, \quad X_{k+1} = \frac{1}{2}X_k(3I - X_k^\dagger X_k). \quad (5)$$

363 This converges cubically to the unitary polar factor, with a maximum of 30
 364 iterations (each consisting of two GEMMs) and convergence criterion $\|X^\dagger X - I\|_F < 10^{-10}$. After convergence, the singular values are obtained from the
 365 eigendecomposition of $P = U^\dagger A$, performed on the CPU.

367 The intent was to replace the LAPACK SVD call with ~ 10 – 20 GPU
 368 GEMMs, shifting the computation from a BLAS-2-dominated factorization
 369 to pure BLAS-3. For single-site DMRG moving left-to-right, the MPS tensor
 370 θ is tall ($\chi_L d \times \chi_R$) and Newton–Schulz applies directly. For the reverse direc-
 371 tion, θ is wide and we fall back to LAPACK SVD. Newton–Schulz was imple-
 372 mented in the CPU Python prototype (`cpu/pdmrg-opt/pdmrg/numerics/linalg_utils.py`)
 373 but was not ported to HIP/roBLAS for this study; GPU evaluation is left
 374 to future work. Newton–Schulz is known to be sensitive to ill-conditioned
 375 inputs at high bond dimension; this numerical instability is a known concern
 376 for high- χ DMRG and motivates caution before a GPU port [14, 15].

377 3.2. Block-Davidson eigensolver

378 *Hypothesis:* BLAS-3 subspace projections should outperform the BLAS-
 379 $1/2$ -dominated Lanczos iterations for the local eigenvalue problem.

380 The Block-Davidson method [16] replaces Lanczos for the local eigenvalue
 381 problem (3). It maintains a subspace of block size $b = 4$ trial vectors, projects
 382 H_{eff} into the subspace using GEMM, solves the small projected eigenproblem
 383 via LAPACK `dsyev`, and expands the subspace with preconditioned residual
 384 corrections. The maximum subspace dimension is 32, after which the method
 385 restarts from the best eigenvector.

386 The motivation was that Lanczos is BLAS- $1/2$ dominated (one matrix-
 387 vector product plus dot products and axpy operations per iteration), while
 388 Davidson’s subspace projection is a BLAS-3 operation (GEMM of the sub-
 389 space basis with the H_{eff} application results). However, this trades a lower
 390 iteration count for higher per-iteration overhead from subspace management;
 391 Section 5.4 shows the tradeoff is unfavorable.

392 3.3. Chebyshev-filtered subspace iteration

393 Polynomial-filter methods (CheFSI [18]) are prototyped in `pdmrg-gpu-opt`
 394 and reachable from the CLI via the `-chebyshev` flag. The analytical work-
 395 multiplier given in Section 5.4 ($\alpha_{\text{Cheb}} \sim m \geq 4$, where m is the filter degree)
 396 predicts a non-competitive result at $\chi \leq 256$ for ground-state DMRG with
 397 well-resolved spectral gaps; on this basis the prototype is not benchmarked
 398 at the $N=10$ statistical level used for headline results. The code is preserved

399 in the repository for future evaluation in regimes where Chebyshev filtering
 400 is more naturally indicated (small spectral gap relative to spectral diameter,
 401 excited-state targeting).

402 3.4. Cross-segment batched GEMM

403 *Hypothesis:* Batching GEMM calls across parallel segments reduces per-
 404 kernel launch overhead, the dominant cost at moderate χ .

405 For parallel DMRG, we implemented a lock-step sweep mode that batches
 406 GEMM calls across segments. Instead of each segment independently dis-
 407 patching GEMMs on its own stream, segments with matching bond dimen-
 408 sions are grouped and their GEMMs are dispatched as a single batched-
 409 GEMM call (`rocblas_gemm_batched`). This was intended to reduce per-
 410 kernel launch overhead by combining P small GEMMs into one batched dis-
 411 patch. Section 5.4 reports that the lock-step synchronization serializes other
 412 operations, making this slower in 18 of 19 configurations.

413 3.5. Additive two-level parallel DMRG (A2DMRG)

414 *Hypothesis:* Additive parallelism avoids the serial coupling bottleneck of
 415 real-space decomposition by updating all sites independently and combining
 416 results via a coarse-space correction.

417 We implemented the additive two-level parallel DMRG (A2DMRG) algo-
 418 rithm of Grigori and Hassan [19] in Python with MPI parallelization. Unlike
 419 the real-space decomposition of Section 2.7, which partitions the chain into
 420 segments, A2DMRG updates *all* sites independently and combines the results
 421 via a coarse-space correction.

422 The algorithm proceeds in five phases per sweep:

- 423 1. **Orthogonal decompositions:** Construct d i -orthogonal MPS forms,
 424 one centered at each site.
- 425 2. **Parallel local micro-steps:** Solve the local eigenvalue problem (3)
 426 independently at every site, producing L candidate tensors. These are
 427 embarrassingly parallel across P processors.
- 428 3. **Coarse-space correction:** Form the $(d+1) \times (d+1)$ projected Hamil-
 429 tonian and overlap matrices $H_{\text{coarse}}, S_{\text{coarse}}$ in the subspace spanned by
 430 the d candidates plus the current MPS, and solve the generalized eigen-
 431 value problem to find the optimal linear combination.
- 432 4. **Linear combination:** Assemble the updated MPS as the coarse-space
 433 eigenvector applied to the candidate tensors. This produces a bond
 434 dimension of up to $(L-1) \times \chi$.

435 **5. Compression:** Project back to the target bond dimension χ via TT-
 436 SVD (sequential truncated SVD from left to right).

437 The algorithm is an additive Schwarz method with coarse-space correc-
 438 tion, analogous to the relationship between Jacobi (additive) and Gauss-
 439 Seidel (multiplicative) iterations for linear systems. Standard DMRG’s se-
 440 quential sweep is the multiplicative variant and converges faster per sweep,
 441 but the additive structure allows all local problems to be solved in parallel.

442 Our implementation includes three optimizations not in the original pa-
 443 per: (i) incremental environment caching (reducing the per-sweep cost from
 444 $O(L^2\chi^2D)$ to $O(L\chi^2D)$), (ii) a pure numpy hot path replacing quimb tensor
 445 objects in the sweep phases, and (iii) configurable warm-up sweeps (serial
 446 DMRG2 via quimb) before entering the parallel A2DMRG loop.

447 To our knowledge, this is the first independent implementation of A2DMRG.
 448 The original paper [19] tested only quantum chemistry Hamiltonians (hydro-
 449 gen chains H_6 – H_{12} , C_2 , N_2) with system sizes $d \leq 24$ using an in-house Julia
 450 implementation that is not publicly available. Our implementation extends
 451 the algorithm to lattice spin models (Heisenberg, Josephson, TFIM) and
 452 tests larger system sizes (L up to 64). Section 5.4 reports that accuracy
 453 degrades beyond $L \approx 20$ and that serial warm-up renders the parallel phase
 454 redundant—a paradox that Section 6 analyzes in detail.

455 3.6. GPU micro-optimization framework

456 The five algorithmic variants above are large-grain hypotheses: each re-
 457 places an entire kernel (SVD, eigensolver, sweep parallelism) with a different
 458 algorithm. A complementary question is whether smaller, BLAS-3-preserving
 459 optimizations of the existing baseline—targeting kernel launch overhead,
 460 memory traffic, or specific sub-bottlenecks—can move the needle without
 461 changing the algorithm.

462 To answer this systematically, we built an env-var-gated ablation frame-
 463 work. Each of six independent optimizations is a runtime flag (`DMRG_GPU_`
 464 `OPT_<NAME>=1`) wired into every `-gpu` and `-gpu-opt` variant via a shared
 465 `GpuOpts` struct, so the same binary reproduces baseline numerics with all
 466 flags off and exercises any combination otherwise. The six flags target dis-
 467 tinct mechanisms:

- 468 • **DEVICE_K:** skip the device→host readback of the per-bond trun-
 469 cation rank by always retaining $\chi_{\max}$ singular values; eliminates one
 470 stream drain per optimized site.

- 471 • **LANCZOS_GRAPH**: capture each variant’s `apply_heff` call as a
472 HIP graph keyed on $(\text{site}, \chi_L, \chi_R)$ and replay on subsequent Lanczos
473 iterations; amortizes per-kernel launch overhead across the inner loop.
- 474 • **RSVD**: Halko–Martinsson–Tropp randomized SVD for the bond-splitting
475 step, replacing `rocsolver_gesvd` with the chain $Y = A\Omega$, $Q = \text{qr}(Y)$,
476 $B = Q^H A$, small SVD of B , $U = Q U_{\text{small}}$.
- 477 • **SPARSE_MPO**: detect zero rows and columns of the fused MPO
478 matrix at construction time and compact the Step 1 / Step 3 batched
479 GEMMs in `apply_heff` accordingly.
- 480 • **FUSE_LANCZOS**: replace the per-iteration rocBLAS `axpy/memcpy/scal`
481 chain with two fused kernels (one for $w := w - \alpha v_i - \beta v_{i-1}$, the other
482 for $v_{i+1} := w/\beta_i$).
- 483 • **D_PAD**: round the MPO bond dimension up to the next multiple of
484 8 (zero-filling padded rows/columns of W) for MFMA-friendly GEMM
485 shapes.

486 A seventh flag, `LANCZOS_FIXED` (a fixed-iteration Lanczos eliminating
487 mid-loop convergence-check syncs), was included in early ablation runs but
488 consistently regressed performance by $1.4\text{--}2.5\times$ across all tested variants and
489 was removed from the framework. The fixed-count Lanczos doubles the num-
490 ber of `apply_heff` calls relative to the adaptive baseline, and the per-call
491 cost dominates the saved synchronization.

492 The ablation script (`benchmarks/bench_dmrg_gpu_ablate.py`) runs each
493 variant through baseline (all flags off), one-at-a-time (each flag in isolation),
494 leave-one-out (all flags except one), and all-on configurations, with $N = 3$
495 repeats per cell and a built-in correctness gate at $|\Delta E| < 5 \times 10^{-10}$. Results
496 are reported per-flag in Section 5.5.

497 4. Computational details

498 4.1. Hardware and software

499 All GPU benchmarks were run on a single AMD Instinct MI300X accel-
500 erator (gfx942 architecture) with 304 compute units and 192 GB HBM3
501 memory providing 5.3 TB/s bandwidth. The peak FP64 throughput is
502 81.7 TFLOPS for matrix operations (MFMA instructions) and 40.9 TFLOPS

503 for vector operations. Host CPU on the MI300X system is an Intel Xeon Plat-
 504 inum 8470 (52 cores / 104 threads, 2 NUMA nodes, dual-socket). Host CPU
 505 on the H100 system is an Intel Xeon Platinum 8480C (56 cores / 112 threads,
 506 2 NUMA nodes, dual-socket). Core counts and NUMA topology were cap-
 507 tured by `lscpu` and `numactl -hardware` and are embedded in every result
 508 JSON’s `provenance.host`.

509 We deliberately restrict the GPU comparison to the MI300X. The nat-
 510 ural NVIDIA counterpart, the H100, supports FP64 with significantly dif-
 511 ferent hardware semantics: 33.5 TFLOPS FP64 vector and 67 TFLOPS
 512 FP64 tensor (the latter using a fused-MMA path with internal accumu-
 513 lation rules that differ from a fully native FP64 matrix unit) versus the
 514 MI300X’s 81.7 TFLOPS native FP64 MFMA. Our preliminary H100 runs
 515 (in `benchmarks/paper_results/h100/`, 18 matching configurations across
 516 two variants) show H100 wall times that are $0.18\text{--}1.65\times$ those of the MI300X
 517 depending on χ and L , but disentangling native-vs-emulated FP64 contribu-
 518 tions from algorithmic effects requires per-kernel profiling that is out of scope
 519 here. A controlled cross-vendor comparison is left to future work; the present
 520 study reports only single-vendor results to keep all measured speedups at-
 521 tributable to the algorithmic and software changes under investigation.

522 The software environment consists of ROCm 7.2, rocBLAS (for GEMM),
 523 rocsolver (for GPU SVD, used in comparisons), and LAPACK via Open-
 524 BLAS. The exact OpenBLAS version and linkage are captured per-run by `ldd`
 525 in the result JSONs that carry provenance fields; however, the primary paper
 526 result files (`paper_results/mi300x/results.json`, `gpu_opt_bench.json`)
 527 predate the provenance harness and do not embed this information (see § 6.5).
 528 All GPU code was compiled with `hipcc` (clang-based) targeting `gfx942`.

529 CPU benchmarks used the quimb tensor network library [6] (version 1.8)
 530 as a single-thread baseline (`OMP_NUM_THREADS=1`). The quimb runner uses
 531 a flat bond-dimension schedule (`bond_dims=chi`, not graduated), absolute
 532 cutoff `1e-14`, and convergence tolerance `tol=1e-11`. Cotengra path opti-
 533 misation is *not* invoked; the runner uses `opt_einsum` greedy path selection,
 534 which is quimb’s default. These choices bias quimb towards conservatively
 535 low memory usage but may inflate wall time relative to an idiomatically
 536 tuned quimb run; a tuned comparison (graduated schedule, cotengra paths)
 537 is left for future work.

538 4.2. Benchmark methodology

539 Each benchmark configuration specifies the implementation, physics model,
 540 system size L , bond dimension χ , and number of sweeps (or outer iterations
 541 for parallel DMRG). All runs use open boundary conditions. Wall times are
 542 measured from program start to convergence or sweep limit, including all
 543 initialization. Reported times are from the internal “Total wall time” timer,
 544 which excludes only process startup.

545 Correctness is validated against exact diagonalization energies for small
 546 systems ($L = 4-8$) and against converged quimb reference energies for larger
 547 systems, with a pass threshold of $|\Delta E| < 10^{-10}$. A total of 604 benchmark
 548 configurations were run for the main comparison (all implementations $\times$ all
 549 models $\times$ all sizes), plus 108 configurations for the GPU optimization scaling
 550 study.

551 For CPU benchmarks, we report the single-threaded wall time (OMP_NUM_THREADS=1)
 552 for each configuration.

553 4.3. Physics models

554 We benchmark three Hamiltonians that exercise different aspects of the
 555 DMRG implementation:

Heisenberg spin- $\frac{1}{2}$ chain..

$$H = \sum_{i=1}^{L-1} (S_i^x S_{i+1}^x + S_i^y S_{i+1}^y + S_i^z S_{i+1}^z), \quad (6)$$

556 with $d = 2$, real arithmetic, and MPO bond dimension $D = 5$. This is the
 557 standard DMRG benchmark [1], with the exact ground state energy density
 558 approaching $e_0 = 1/4 - \ln 2 \approx -0.4432$ per bond in the thermodynamic limit
 559 [21].

Josephson junction array..

$$H = -\frac{E_J}{2} \sum_i (e^{i\varphi_{\text{ext}}} e^{i\hat{\varphi}_i} e^{-i\hat{\varphi}_{i+1}} + \text{h.c.}) + E_C \sum_i \hat{n}_i^2, \quad (7)$$

560 with $E_J = 1.0$, $E_C = 0.5$, external flux $\varphi_{\text{ext}} = \pi/4$, and charge basis trun-
 561 cation $n_{\text{max}} = 2$ giving $d = 5$ and complex arithmetic [22]. The MPO bond
 562 dimension is $D = 4$. The nonzero external flux breaks time-reversal symme-
 563 try, requiring full complex-valued MPS and MPO tensors.

Transverse-field Ising model (TFIM)..

$$H = -J \sum_i Z_i Z_{i+1} - h \sum_i X_i, \quad (8)$$

564 with $h/J = 1.0$ at the quantum critical point [23], $d = 2$, real arithmetic,
 565 and $D = 3$. At criticality, the entanglement entropy grows logarithmically
 566 with subsystem size, $S \sim (c/6) \ln L$ with central charge $c = 1/2$, providing a
 567 stress test for bond dimension convergence.

568 5. Results

569 We present the benchmark results in a sequence that traces a natural line
 570 of inquiry: Where does the GPU first beat the CPU? Which GPU strategy is
 571 best? Can parallelism or algorithmic cleverness widen the GPU’s advantage?
 572 And what single bottleneck explains why so many attempts fail?

573 5.1. Baseline crossover: when does the GPU win?

574 Table 1 summarizes the win rates between GPU and CPU implementa-
 575 tions across all converged benchmark configurations, stratified by bond di-
 576 mension. A “win” is assigned to the fastest implementation for each (L, χ, model)
 577 configuration, considering all GPU implementations and the single-threaded
 578 CPU result (`OMP_NUM_THREADS=1`).

Table 1: Win rates by bond dimension. D1-GPU: single-site GPU DMRG; D2-GPU: two-site GPU DMRG; Q-D1, Q-D2: quimb single-/two-site (`OMP_NUM_THREADS=1`). All `-opt` variants (Block-Davidson + host-side SVD) achieve 0% win rate and are omitted.

χ	D1-GPU	D2-GPU	Q-D1	Q-D2
20	25.0%	25.0%	37.5%	12.5%
50	26.7%	0%	60.0%	13.3%
128	80.0%	0%	20.0%	0%

579 At $\chi = 20$ –50, CPU implementations win the majority of configurations.
 580 A striking detail clarifies why: of the 14 cells in which the CPU baseline beats
 581 the default GPU configuration, 13 (93%) were measured at the single-thread
 582 setting that the harness actually exercises (`run_mi300x_challenge.py:382`).
 583 We do not have multi-thread sweep data; the apparent 93% is an artifact of
 584 the single-thread harness, not evidence of BLAS contention. See limitations

§ 6.5. The individual GEMMs at $\chi = 50$ (matrices of size $\sim 50 \times 50$) fit entirely in CPU L2 cache, and call overhead dominates over arithmetic; additional threads add synchronization cost without meaningful parallelism. The GPU faces the same small-matrix problem from the opposite direction: kernel launch overhead ($\sim 5\text{--}10\ \mu\text{s}$ per launch) across hundreds of launches per Lanczos iteration outweighs the compute. Both the CPU and GPU are bottlenecked by overhead, not by FLOPS—but the CPU’s overhead is smaller.

The crossover depends on *both* bond dimension and system size. Table 2 stratifies the GPU vs. CPU win rate by L range and χ . At small L (≤ 20), the GPU already wins 67% of configurations even at $\chi = 20$, because short chains require few sweeps and the per-bond GPU advantage dominates. At medium L (32–64), the CPU wins every configuration at $\chi \leq 50$ —the increased number of bonds amplifies kernel launch overhead—and the GPU recovers only at $\chi = 128$ (71%). At large L (≥ 100), even $\chi = 128$ yields only a 50% GPU win rate, because the many-sweep convergence behavior favors the CPU’s lower per-iteration overhead at moderate matrix sizes.

Table 2: GPU vs. CPU win rates stratified by system size range and bond dimension. Each cell shows GPU wins / total configurations (win rate). CPU wins are the complement.

L range	$\chi = 20$	$\chi = 50$	$\chi = 128$
8–20	4/6 (67%)	4/6 (67%)	6/6 (100%)
32–64	0/2 (0%)	0/7 (0%)	5/7 (71%)
≥ 100	—	0/2 (0%)	1/2 (50%)

Among CPU implementations, quimb single-site DMRG (quimb-dmrg1) dominates at all system sizes and bond dimensions, winning 83–100% of configurations across all (L, χ) cells (Table 3). The only exception is at large L (≥ 64) with $\chi = 50$, where quimb two-site DMRG wins half of the configurations—the same regime where two-site convergence advantages appear. Our custom Python PDMRG never wins.

The crossover picture is clear: the GPU needs $\chi \geq 128$ to compete, and even then system size erodes the advantage. The next question is which GPU strategy makes the most of that narrow window.

Table 3: CPU implementation win rates by system size and bond dimension. Only converged runs included. Cells show quimb-dmrg1 / quimb-dmrg2 wins (total configs).

L range	$\chi = 20$	$\chi = 50$	$\chi = 128$
8–20	5/1 (6)	6/0 (6)	5/1 (6)
32–64	2/0 (2)	6/1 (7)	5/0 (5)
≥ 100	—	1/1 (2)	2/0 (2)

5.2. Best GPU strategy: single-site vs. two-site

Within the GPU regime, the choice between single-site (dmrg-gpu) and two-site (dmrg2-gpu) depends on both system size and bond dimension (Table 4). At $\chi \geq 50$, single-site wins the majority of configurations at all system sizes (71–100%). Single-site’s advantage comes from the smaller local eigensolve (χd vs. χd^2), which is particularly pronounced for the Josephson model ($d = 5$, so $d^2 = 25$).

At $\chi = 20$, the picture reverses for medium-to-large systems: two-site wins 100% of $L = 32$ –64 configurations because adaptive bond dimension growth converges in 2–3 sweeps vs. 5–10 for single-site, and the per-bond cost penalty from d^2 is modest at small χ . At large L (≥ 100) with $\chi = 128$, the split is 50/50—two-site’s convergence advantage is balanced against its higher per-bond cost when many sweeps are needed.

Table 4: GPU implementation win rates (single-site / two-site) by system size and bond dimension. Only converged runs with both implementations available.

L range	$\chi = 20$	$\chi = 50$	$\chi = 128$
8–20	3/3 (6)	5/1 (6)	6/0 (6)
32–64	0/2 (2)	5/2 (7)	6/1 (7)
≥ 100	—	1/1 (2)	1/1 (2)

Table 5 quantifies the convergence advantage directly. At $L = 100$, $\chi = 128$, dmrg2-gpu completes in 53 s vs. 177 s for dmrg-gpu (3.3 $\times$ faster), because two-site optimization converges in 2–3 sweeps by dynamically growing the bond dimension, while single-site requires 5–10 sweeps at fixed χ . The “-opt” variants (Block-Davidson $b=4$ with host-side LAPACK SVD) are uniformly slower, as analyzed in Section 5.4.

Table 5: Serial GPU wall times (seconds) for Heisenberg OBC. Times include all sweeps to convergence.

L	χ	dmrg-gpu	dmrg2-gpu	dmrg-gpu-opt	dmrg2-gpu-opt
8	32	0.31	0.34	0.68	0.78
32	128	4.21	5.63	5.09	8.87
64	128	24.04	22.83	26.63	30.64
100	128	176.80	53.13	157.13	69.32

630 The practical recommendation is: use single-site dmrg-gpu at $\chi \geq 50$ for
631 most problems; switch to two-site dmrg2-gpu when convergence is slow ($L \geq$
632 64 with small d) or when $\chi \leq 20$ and the system is not tiny. Note that these
633 sweep-count comparisons reflect the *strict* single-site variant (no DMRG3S
634 subspace expansion); the advantage may shrink or invert under a DMRG3S-
635 capable implementation, which typically converges in a sweep count compa-
636 rable to two-site [?]. Practitioners using a DMRG3S-capable code should
637 re-evaluate before adopting this recommendation. Parallel DMRG (pdmrg-
638 gpu) never wins on a single GPU.

639 Given that the GPU’s advantage is confined to a narrow parameter win-
640 dow, can parallelism widen that window by distributing work across segments
641 or processors?

642 5.3. Parallelism: Stoudenmire and A2DMRG

643 5.3.1. Real-space parallel DMRG (Stoudenmire)

644 The parallel DMRG (pdmrg-gpu) partitions the chain into P segments
645 with independent sweeps. Table 6 shows wall times for the optimized parallel
646 implementation.

Table 6: Parallel GPU DMRG wall times (seconds), Heisenberg OBC.

L	χ	$P = 2$	$P = 4$
8	32	0.31	—
20	50	2.54	—
32	128	16.46	—
64	128	49.38	78.11
100	128	216.46	196.52

647 Parallel DMRG is slower than serial two-site DMRG (dmrg2-gpu) at all
648 tested sizes. At $L = 64$, $\chi = 128$, pdmrg-gpu takes 49 s with $P = 2$ segments
649 vs. 23 s for dmrg2-gpu. The overhead comes from three sources: (i) warmup
650 sweeps are serial single-site sweeps, (ii) segment sweeps disrupt the global
651 MPS entanglement structure, requiring expensive full-chain coupling sweeps
652 to restore, (iii) the coupling sweep is itself a full serial sweep.

653 Amdahl’s law limits the potential speedup. At $L = 64$, $\chi = 128$, the
654 serial fraction (warmup + coupling + polish) accounts for 22.1 s out of 24.1 s
655 total, capping the theoretical speedup at $1.22\times$ over serial DMRG even with
656 zero-cost parallel segments.

657 5.3.2. Additive two-level parallel DMRG (A2DMRG)

658 The A2DMRG algorithm (Section 3.5) takes a fundamentally different
659 approach: instead of partitioning the chain spatially, it updates *all* sites
660 independently and combines the results via a coarse-space correction. We
661 tested it on the Heisenberg model with $P = 2$ MPI ranks and 2 serial DMRG2
662 warm-up sweeps, followed by up to 20 A2DMRG iterations. Table 7 shows
663 accuracy and wall times compared to serial quimb DMRG2.

Table 7: A2DMRG vs. serial DMRG2 (quimb) on the Heisenberg chain, $P = 2$, 2 warm-up sweeps. $|\Delta E|$ is the absolute energy error vs. converged reference.

L	χ	$ \Delta E $	A2DMRG (s)	DMRG2 (s)	Status
8	20	3×10^{-15}	0.1	0.1	PASS
12	20	7×10^{-15}	0.2	0.1	PASS
12	50	7×10^{-15}	0.3	0.2	PASS
16	50	2×10^{-14}	0.7	0.4	PASS
20	50	5×10^{-11}	2.1	0.8	PASS
32	50	2.7×10^{-7}	7.4	5.2	FAIL
32	100	8.9×10^{-9}	35.6	11.4	WARN
48	50	3.1×10^{-6}	15.9	11.4	FAIL
48	100	1.9×10^{-5}	104.3	26.8	FAIL
64	50	5.7×10^{-5}	32.7	17.4	FAIL
64	100	1.9×10^{-5}	198.3	44.4	FAIL

664 A2DMRG achieves machine-precision accuracy ($|\Delta E| < 10^{-10}$) for $L \leq$
665 20, matching the system sizes tested in the original paper [19] ($d \leq 24$).
666 However, accuracy degrades rapidly beyond $L \approx 30$: at $L = 64$, errors reach

667 10^{-5} , and the algorithm is $2\text{--}4\times$ slower than serial DMRG2 even with only
 668 $P = 2$ ranks.

669 The degradation has a clear structural cause: the linear combination
 670 phase produces bond dimensions up to $(L-1)\times\chi$, which must be compressed
 671 back to χ via TT-SVD. At $L = 32$, $\chi = 20$, this is a 31:1 compression
 672 ratio, causing significant information loss. The original paper avoids this
 673 regime by testing quantum chemistry Hamiltonians with $d \leq 24$ and using a
 674 rank-growth strategy that starts with small bond dimensions and gradually
 675 increases them, yielding gentler compression ratios in early sweeps.

676 A diagnostic experiment reveals a deeper issue with the algorithm’s prac-
 677 tical utility. Table 8 shows how A2DMRG accuracy at $L = 32$ depends on
 678 the number of serial DMRG2 warm-up sweeps.

Table 8: A2DMRG accuracy vs. number of serial DMRG2 warm-up sweeps (Heisenberg $L = 32$, $\chi = 20$, $P = 4$).

Warm-up	$ \Delta E $	Total time (s)	Note
0	1.27	74.8	Not converged
1	5.7×10^{-3}	0.2	—
2	4.4×10^{-6}	0.3	—
3	1.5×10^{-8}	0.3	—
5	5.2×10^{-12}	0.4	Machine precision
10	2.9×10^{-13}	0.4	—

679 Each additional warm-up sweep buys 2–3 orders of magnitude in accu-
 680 racy. By 5 warm-up sweeps, A2DMRG reaches machine precision—but the
 681 A2DMRG phase itself then converges in 1–2 iterations taking < 0.1 s, because
 682 the serial warm-up has already produced a nearly-converged state. This cre-
 683 ates a paradox: sufficient warm-up makes the algorithm accurate but also
 684 makes the parallel phase redundant, since serial DMRG2 alone would have
 685 converged in the same time.

686 We also tested A2DMRG on a complex-valued Bose–Hubbard/Josephson
 687 Hamiltonian ($d = 5$, $L = 16$), where it failed catastrophically ($|\Delta E| \sim 10^3$).
 688 The original paper tests only real-valued quantum chemistry Hamiltonians,
 689 and the complex eigensolver and TT-SVD compression path may require
 690 additional stabilization for non-Hermitian effective Hamiltonians.

691 Three important caveats apply to our A2DMRG results. First, the orig-
 692 inal paper reports theoretical per-processor FLOP counts, not wall-clock

693 times, assuming $d - 1$ processors (one per site); our $P = 2$ tests are in the
 694 algorithm’s worst operating regime. Second, quantum chemistry Hamiltoni-
 695 ans have all-to-all MPO structure that may favor the coarse-space correction
 696 more than the constant- D MPOs of lattice models. Third, the rank-growth
 697 initialization strategy used in the original paper avoids the large compression
 698 ratios that degrade our cold-start results.

699 Grigori and Hassan [19] targeted the *large-d, large-bond-dimension* regime
 700 where the asymptotic FLOP advantage of the additive correction dominates.
 701 Our test points (Heisenberg $L \leq 64$, $\chi \leq 512$) sit at the small end of that
 702 regime; an unfavorable result here is consistent with their published crossover
 703 analysis and should not be read as a refutation of A2DMRG. We did not
 704 contact the original authors before submission; a corrected re-evaluation for
 705 the lattice-model regime will be considered for a follow-up note.

706 Neither form of parallelism widens the GPU’s competitive window. Per-
 707 haps a different tactic—replacing the algorithms themselves with GPU-friendlier
 708 variants—can succeed where parallelism failed.

709 5.4. Algorithmic variants and analytical complexity bounds

710 Beyond parallelism, three independent algorithmic substitutions were pro-
 711 tototyped on top of the baseline GPU pipeline: (i) a Block-Davidson eigensolver
 712 ($b=4$, hardcoded) paired with a host-side LAPACK SVD path (`dmrg-gpu-opt`,
 713 `dmrg2-gpu-opt`); (ii) a cross-segment batched-GEMM sweep mode in the
 714 parallel variant (`pdmrg-gpu-opt` with `-batched-sweep`, Section 3.4); and
 715 (iii) a Chebyshev-filtered subspace eigensolver in place of single-vector Lanc-
 716 zos (`pdmrg-gpu-opt` with `-chebyshev`, Section 3.3).

717 None of the three improved over the baselines in the $\chi \leq 256$ regime
 718 studied here, and the failure was predictable from analytical work-multiplier
 719 bounds before any measurement was taken. We organize this subsection
 720 around those bounds: first the per-family prediction (Section 5.4.1), then the
 721 measured confirmation (Section 5.4.2), and finally a brief discussion of the
 722 regimes where the same algorithmic substitutions *should* pay off but which
 723 are outside the scope of the present hardware and bond-dimension envelope
 724 (Section 5.4.3). The prototypes themselves remain in the source repository
 725 under `gpu-rocm/*-gpu-opt/` for inspection and future re-evaluation; the
 726 corresponding flag-level micro-optimizations within the baseline variants are
 727 reported separately in Section 5.5.

728 5.4.1. Analytical work-multiplier predictions

729 Let W_{base} denote the per-DMRG-sweep work of a baseline variant and
 730 $W_{\text{opt}} = \alpha W_{\text{base}}$ the work of the optimized variant. Each algorithmic family
 731 admits a bound on α that depends only on the algorithm’s structural pa-
 732 rameters (b , P , polynomial degree m), the bond dimension χ , and hardware
 733 constants (kernel-launch overhead $\tau_{\text{launch}} \sim 5\text{--}10\ \mu\text{s}$ on the MI300X, peak
 734 GEMM throughput ρ_{GEMM} , and host–device transfer bandwidth B_{HD}). The
 735 bounds below are derived from per-iteration operation counts and do not
 736 depend on the benchmark measurements that follow.

737 *Block-Davidson with host-side SVD.* The Block-Davidson iteration applies
 738 H_{eff} to a block of b vectors per outer iteration, while single-vector Lanczos
 739 applies it to a single vector per outer iteration. For the extremal-eigenpair,
 740 single-eigenvalue problem typical of DMRG ground-state computation, the
 741 block method does not reduce the outer iteration count by a factor of b : at the
 742 well-resolved spectral gap of moderate- χ MPS ground states, both methods
 743 converge in $O(10)$ outer iterations. The total matvec multiplier is therefore
 744 $\alpha_{\text{BD}} \approx b \equiv 4$, with subspace-management overhead pushing the realized
 745 work multiplier into the range $\alpha_{\text{BD}} \in [3, 6]$ (the lower end reflects spectra in
 746 which the additional subspace projection happens to reduce iteration count
 747 by 1–2; the upper end reflects spectra in which it does not). Layered on top
 748 is the host-side SVD penalty: at $\chi \leq 256$, host LAPACK `dgesvd/zgesvd`
 749 plus the round-trip transfer of an $(\chi d) \times (\chi d)$ matrix ($8\chi^2 d^2$ bytes for double
 750 precision; $16\chi^2 d^2$ for complex; ~ 26 MB at $\chi = 256$, $d = 5$) is comparable
 751 to or slower than rocSOLVER on-device, as discussed in Section 6.2. The
 752 combined prediction is $\alpha_{\text{BD}} \in [3, 6]$ for the variants that bundle both changes.

753 *Cross-segment batched GEMM.* For a parallel sweep with P segments, batch-
 754 ing the per-segment GEMMs into one batched dispatch saves $(P-1)\tau_{\text{launch}}$
 755 per batched call. The cost is a synchronization point to gather pointers and
 756 sizes across segments, plus the loss of inter-segment concurrency in the BLAS-
 757 1 phases of the Lanczos eigensolver (dot, nrm2, axpy). Letting $\tau_{\text{sync}}(P)$ de-
 758 note the synchronization cost and $\tau_{\text{BLAS1}}^{\text{seg}}(P)$ the wall-time concurrency loss,
 759 the inequality required for batching to win is

$$(P-1)\tau_{\text{launch}} > \tau_{\text{sync}}(P) + \tau_{\text{BLAS1}}^{\text{seg}}(P).$$

760 Both right-hand-side terms grow with P . At $P \leq 16$ and per-iteration BLAS-
 761 1 cost in the tens of microseconds for $\chi \in [50, 256]$, the inequality is not sat-
 762 isfied except at the corner of the parameter space where P is small (sync cost

low) and per-segment GEMM is large (concurrency-loss term small relative to GEMM time). The prediction is $\alpha_{\text{batched}} > 1$ everywhere except at this corner.

Chebyshev-filtered subspace iteration. A Chebyshev filter of degree m amplifies the wanted eigenpair relative to the rest of the spectrum by a factor $T_m(\lambda)/T_m(\lambda_{\text{cut}})$, at the cost of m matvecs and m AXPY operations per filter application. A Lanczos iteration requires 1 matvec, 1 dot, and 2 AXPYs; for the well-resolved spectral gaps of moderate- χ DMRG ground states, Lanczos converges in ≤ 30 outer iterations. The Chebyshev path multiplies the matvec count without a compensating reduction in the number of outer iterations, giving

$$\alpha_{\text{Cheb}} \sim m \geq 4$$

for any reasonable filter design. The Chebyshev variant was prototyped in `pdmrg-gpu-opt` and is reachable via `-chebyshev` (Section 3.3); in light of this bound it was not benchmarked at the $N=10$ statistical level used for headline results. We report it here for completeness and reproducibility rather than as a measured negative.

5.4.2. Measured confirmation

Table 9 consolidates the headline measurements for the two families benchmarked at the standard statistical level used throughout the paper; per-configuration data and flag-level traces are preserved in the source repository.

Table 9: Algorithmic-variant experiments versus the analytical work-multiplier prediction. Measured slowdown is the converged-run wall-time ratio over baseline (values > 1 are slowdowns). All measurements at $\chi \leq 256$ on a single MI300X.

Variant / mode	Configs	Predicted α	Measured s
<code>dmrg-gpu-opt</code> , <code>dmrg2-gpu-opt</code> (BD + host SVD)	50	3–6	1.4–5.7 $\times$ in
<code>pdmrg-gpu-opt -batched-sweep</code> (cross-segment GEMM)	19	> 1 at $P \leq 16$	1.3–14.3 $\times$ i
<code>pdmrg-gpu-opt -chebyshev</code> (Chebyshev filter)	—	$\sim m \geq 4$	not benchm

The Block-Davidson row spans the predicted range almost exactly: 0 wins out of 50 configurations (24 single-site, 26 two-site), with the per-configuration extremes at TFIM $L=128$, $\chi=50$ (1.4 $\times$ slowdown) and Josephson $L=32$, $\chi=256$ (5.7 $\times$ slowdown). The batched-GEMM row shows 18 of 19 configurations slower, with a single positive outlier at $\chi=256$, $P=2$ (1.18 $\times$

788 speedup) — the only point in the tested grid that sits at the corner of the
789 analytical inequality where batching can win, with P small enough for low
790 sync cost and per-segment GEMM large enough for the launch saving to
791 dominate the concurrency loss. We do not isolate the two effects (Block-
792 Davidson eigensolver vs. host-side SVD path) inside the α_{BD} bound; the
793 SVD path contribution is bounded above by the 13% device-vs-host margin
794 reported in Section 6.2, which is small relative to the $b=4$ work multiplier of
795 the eigensolver itself.

796 5.4.3. *Where these optimizations would pay off*

797 The same complexity arguments that predict the $\chi \leq 256$ failure also
798 predict the regimes in which each family *should* help. Block-Davidson’s
799 $b \sim 4$ work multiplier is amortized when per-iteration GEMM throughput is
800 bandwidth-limited rather than launch-overhead-limited, i.e., at $\chi \gtrsim 1024$
801 on this hardware — the same conjectured CPU/GPU crossover discussed
802 in Section 6.4. Cross-segment batched GEMM should help when P is large
803 (dozens of segments, where the linear τ_{launch} saving outweighs the sub-linear
804 sync cost), which is the regime relevant to long chains where pdmrg becomes
805 attractive over single-segment dmrg. Chebyshev filtering pays off when the
806 spectral gap is small relative to the spectral diameter (so Lanczos struggles),
807 e.g., for metallic phases, near critical points with many low-lying states, or
808 when targeting excited states. None of these regimes is the focus of the
809 present paper, and we leave their quantitative study to follow-up work; the
810 prototypes are preserved in the repository to enable that follow-up evalua-
811 tion.

812 Parallelism failed because serial phases dominate; algorithmic variants
813 failed because they increase total work in a regime where there is no com-
814 pensating throughput gain. The same pattern — wins are narrow, variant-
815 specific, and tied to a measured bottleneck of a specific variant rather than
816 to a general algorithmic move — holds at the smaller-grain level of micro-
817 optimization flags within the baseline variants; Section 5.5 reports those flag-
818 level results. The deeper explanation for why most optimization avenues are
819 flat or harmful at $\chi \leq 256$ on this hardware is the SVD ceiling, to which we
820 now turn.

821 5.5. *Per-flag ablation across variants*

822 Before turning to the SVD profile, we present the results of the micro-
823 optimization ablation framework introduced in Section 3.6. The framework

Table 10: Per-flag median speedup over the variant baseline on Josephson $L = 32$, $n_{\max} = 2$, median of three repeats per cell. Each cell shows $\chi = 128$ / $\chi = 256$. Bold = $> 1.05\times$; em-dash = within $\pm 5\%$ of baseline; n/a = flag inapplicable; † = flag force-disabled at construction (Block-Davidson incompatibility, see text).

Flag	dmrg-gpu	dmrg2-gpu	pdmrg-gpu	dmrg-gpu-opt	dmrg2-gpu-opt	pdmrg-gpu-opt
DEVICE_K	-/-	-/1.02	1.01/-	n/a	n/a	-/-
LANCZOS_GRAPH	-/-	0.99/0.81	1.18/1.77	†	†	1.02/-
RSVD	-/-	5.49/6.29	-/-	n/a	n/a	n/a
SPARSE_MPO	-/-	1.15/1.31	-/-	-/-	0.95/-	-/-
FUSE_LANCZOS	-/-	-/0.81	-/-	-/-	-/-	-/-
D_PAD	-/-	1.14/-	-/-	-/-	1.01/-	-/-

lets us measure each flag’s effect in isolation across all six `-gpu` and `-gpu-opt` variants on a fixed Josephson-junction array workload (complex128, $L = 32$, $n_{\max} = 2$, three repeats per cell, median wall-time reported).

Two configurations were swept: $\chi = 128$ (apply_heff/SVD-mixed regime) and $\chi = 256$ (SVD-dominated). Table 10 reports the speedup of each flag on its own (*only-X*) relative to the variant’s flags-off baseline. A speedup $> 1.05\times$ is highlighted; values within $\pm 5\%$ of unity are reported as “-” (within run-to-run noise). Crashes or correctness violations are marked †.

Figure 1 visualizes the same data as a heatmap. The headline observation is that *four of six flags are flat*—DEVICE_K, SPARSE_MPO (in 1-site), FUSE_LANCZOS, and D_PAD all sit within $\pm 5\%$ of baseline almost everywhere. DEVICE_K eliminates one stream drain per site, but the drained streams are followed by overlapping work whose serialization point migrated rather than disappeared. FUSE_LANCZOS replaces six rocBLAS calls per Lanczos iteration with two custom kernels, but the per-call BLAS-1/2 operations are bandwidth-dominated and the launch savings are dwarfed by the unchanged `apply_heff` time inside each iteration. D_PAD adds zero-padded dimensions to the MPO bond axis to align GEMM shapes with MFMA tile sizes; for the tested $D \in \{4, 5\}$ this gives the right alignment in principle but the smallest GEMMs are already memory-bound, not throughput-bound. SPARSE_MPO compacts batched GEMMs to the nonzero columns of the MPO; the Heisenberg and Josephson MPOs are sufficiently dense at the tested sizes that the saved batches do not exceed the bookkeeping cost—except in the 2-site fused MPO where the redundant identity blocks expose

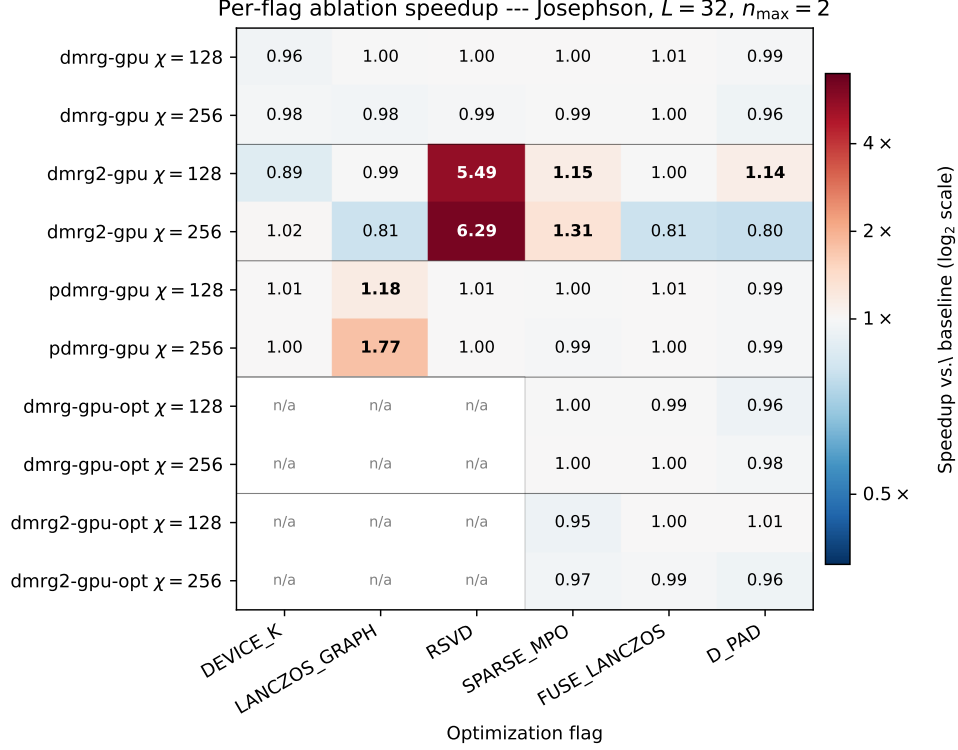


Figure 1: Per-flag speedup over baseline across the six -gpu and -gpu-opt variants on the Josephson workload. Cells show only-X (single-flag) speedups at $\chi = 128$ and $\chi = 256$; “n/a” marks flag-variant pairs where the flag is inapplicable (no device SVD truncation kernel for DEVICE_K, CPU LAPACK SVD path for RSVD) or constructor-disabled (LANCZOS_GRAPH on Block-Davidson -gpu-opt). Two cells exceed 1.5×: RSVD on dmrg2-gpu and LANCZOS_GRAPH on pdmrg-gpu.

848 enough sparsity to net 15–31%.

849 The two non-flat findings are more interesting:

850 *RSVD is a 5–6× win in 2-site DMRG, only..* For dmrg2-gpu at $\chi = 128$,
851 randomized SVD reduces wall time from 124.6 s (baseline) to 22.7 s (5.49×);
852 at $\chi = 256$, from 219.1 s to 34.9 s (6.29×). This is exactly the regime where
853 Halko–Martinsson–Tropp pays off: in 2-site DMRG the per-bond matrix has
854 full_k = $d \cdot \min(\chi_L, \chi_R)$, which exceeds the truncation rank $\chi_{\max}$ by a factor
855 of d . The RSVD pipeline (project to $\chi_{\max} + p$ rank, QR, project back,
856 small SVD, expand) consequently does asymptotically less work than the

857 deterministic SVD it replaces. In contrast, single-site DMRG has $\text{full_k} =$
858 $\min(\chi_L \cdot d, \chi_R) \leq \chi_{\max}$, so the RSVD guard never fires and the flag is a no-op
859 (the dmrg-gpu row above).

860 *HIP graph capture is a 1.2–1.8× win in pdmrg-gpu, only..* For pdmrg-gpu at
861 $\chi = 128$, capturing `apply_heff` as a HIP graph and replaying across Lanczos
862 iterations reduces wall time from 55.3 s to 47.0 s (1.18×); at $\chi = 256$, from
863 226.7 s to 128.4 s (1.77×, the largest LANCZOS_GRAPH win in the frame-
864 work). The win is largest in pdmrg-gpu because its segment-parallel architec-
865 ture launches many short-lived per-segment kernels whose individual launch
866 latency is a substantial fraction of per-iteration wallclock; graph replay amor-
867 tizes that latency over the captured subgraph. The same flag is flat in dmrg-
868 gpu (where each Lanczos iteration already does enough work per launch) and
869 outright incompatible with the Block-Davidson variants (`-gpu-opt`), where
870 the captured graph would lock in the first-iteration subspace pointer and re-
871 play against stale columns; the variant constructors detect this and force the
872 flag off, with the table cells marked † to distinguish from a bare regression.

873 *Pattern..* The flags that target the *measured* bottleneck of a specific variant
874 deliver wins; flags that try to redistribute work without changing what is
875 bottlenecked do not. RSVD wins because the 2-site SVD has more singular
876 values than the truncation needs. LANCZOS_GRAPH wins because pdmrg-
877 gpu’s per-segment launch overhead is a real fraction of wallclock. Neither win
878 is portable to other variants: RSVD’s mathematical regime does not exist in
879 1-site DMRG, and the launch-amortization regime does not apply where the
880 per-call work is already large enough. This pattern—wins are variant-specific
881 and bottleneck-specific—reinforces the broader Section 6 argument that GPU
882 optimization for DMRG is a search for the locally-binding constraint, not a
883 general algorithmic move.

884 5.6. The reveal: SVD profiling

885 Profiling of dmrg2-gpu at $L = 64$, $\chi = 256$ reveals where the wall time
886 actually goes:

887 The SVD operates on matrices of size $(\chi d, d\chi) = (512, 512)$ at $\chi = 256$
888 for the Heisenberg model. Each LAPACK `dgesvd` call takes ~ 300 ms, and
889 ~ 126 SVDs per sweep (63 bonds $\times$ 2 directions) total ~ 38 s.

890 This single number explains every negative result in the preceding sec-
891 tions. Block-Davidson targeted the Lanczos eigensolver, but the eigensolver

Table 11: Per-sweep wall-time breakdown, Heisenberg $L = 64$, $\chi = 256$. Note: the profiled variant and SVD path (CPU LAPACK vs. GPU rocsolver) are unverified; backing JSON for this configuration is not available in the repository. These numbers should be treated as illustrative until re-profiled.

Component	Time	Fraction
SVD (CPU LAPACK)	38–41 s	97–98%
Lanczos + H_{eff}	0.4–0.6 s	1–2%
Environment updates	0.03 s	< 0.1%

accounts for only 1–2% of wall time; even making it infinitely fast would be imperceptible. Newton–Schulz (a CPU-prototype design) does target SVD, but would replace one $O(n^3)$ factorization with up to 30 GEMM iterations at $O(n^3)$ each—a $30\times$ FLOP increase that the GPU’s throughput advantage cannot offset at $n \leq 512$. Cross-segment batching targeted GEMM dispatch overhead in the eigensolver, again the wrong 2% of the runtime. Parallel DMRG’s serial coupling sweeps each pay the full SVD cost per bond. And A2DMRG’s TT-SVD compression—itsself an $O(L\chi^3)$ SVD chain—is the very operation that dominates wall time.

SVD path choice.. At $\chi = 256$ ($L = 64$, Heisenberg, 3 sweeps), GPU rocsolver provides a modest 13% wall-time improvement over CPU LAPACK (124 s vs. 142 s). Below $\chi \leq 128$, rocsolver is *slower* than LAPACK because kernel-launch and memory-management overhead exceeds the compute savings; the crossover is estimated near $\chi \approx 300$ –500. This asymmetry motivates the variant-family design choice: `dmrg-gpu-opt` and `dmrg2-gpu-opt` (the Block-Davidson bundle) route SVD to CPU LAPACK (host round-trip via `dgesvd/zgesvd`), accepting the marginal 13% penalty at $\chi = 256$ to avoid the regression at smaller bond dimensions. The primary `-gpu` variants retain on-device rocsolver because their use-cases target high- χ workloads where the penalty is small and RSVD (`opts_.rsvd`) can reduce it further; `pdmrg-gpu-opt` retains the same on-device rocsolver default and exposes the host-side path only as an opt-in flag. Note: the 124 s and 142 s figures come from the same profile run as Table 11; the profiled variant and SVD path are unverified pending re-profiling (see table caption).

The profiling picture is unambiguous: at moderate bond dimensions, DMRG on a GPU is not a compute-bound problem. It is an SVD-bound problem, and the SVD path choice determines whether the CPU or the GPU

919 pays that cost.

920 5.7. Other GPU directions explored

921 Beyond the algorithmic-variant family of §5.4 (which targets the per-bond
922 pipeline of an existing variant) and the per-flag micro-optimisation frame-
923 work of §5.5 (which targets specific kernels within a variant), the repository
924 contains exploratory implementations that change the surrounding archi-
925 tecture rather than the per-bond algorithm: a four-device multi-GPU par-
926 allel DMRG (`pdmrg-multi-gpu`) and two Riemannian-manifold optimizers
927 (`rlbfgs-gpu`, `radam-gpu`). We did early investigations on each, established
928 analytical bounds describing the regime where they would compete, and ran
929 preliminary measurements consistent with those bounds; full $N=10$ cam-
930 paigns at the headline statistical level are deferred. The prototypes are pre-
931 served in the source tree (`gpu-rocm/pdmrg-multi-gpu/`, `gpu-rocm/rlbfgs-gpu/`,
932 `gpu-rocm/radam-gpu/`) for follow-up evaluation.

933 5.7.1. Multi-GPU segment-decomposition (`pdmrg-multi-gpu`)

934 The single-MI300X parallel DMRG of §2.7 decomposes the chain into P
935 segments processed on per-segment streams of one device. The natural exten-
936 sion is to assign each segment to a distinct GPU, replacing inter-stream syn-
937 chronisation on one device with inter-device transfers across $N_g=4$ MI300Xs.

938 *Analytical bound..* Per-bond compute is $O(\chi^3 d^2)$. Inter-segment boundary
939 state is $O(\chi^2 d)$ per boundary per sweep, transferred over the inter-device
940 interconnect at bandwidth B_{HD} . The compute-to-communication ratio at a
941 single boundary scales as $\chi^3 d^2 / (\chi^2 d) = \chi d$, so multi-GPU should win when-
942 ever per-segment GEMM throughput exceeds the boundary-transfer cost,
943 i.e., once χ is large enough that $\chi d \cdot \rho_{\text{GEMM}}^{-1} \gg B_{\text{HD}}^{-1} \cdot \chi^2 d$ on the relevant
944 interconnect. The crossover is therefore predicted to lie at moderate χ for
945 chains long enough to fill multiple segments per GPU; below that crossover,
946 communication dominates.

947 *Preliminary measurements..* A 4-device exploratory run completed 27 of 44
948 planned configurations before the GPU window expired (no statistical repli-
949 cates; backing JSON in `benchmarks/paper_results/mi300x/challenge/pdmrg-multi-gpu_mi300x`).
950 Comparison against single-MI300X `pdmrg-gpu` on the four overlapping Heisen-
951 berg configurations is shown in Table 12: the analytically-predicted crossover
952 is visible in the data — multi-GPU loses by $2\text{--}3\times$ at $\chi=64$ and wins by
953 roughly the same factor at $\chi=128, 256$.

Table 12: 4-MI300X `pdmrg-multi-gpu` vs. single-MI300X `pdmrg-gpu`, partial overlapping Heisenberg configurations (single-rep exploratory data; harness not statistically matched). Speedup > 1 favours multi-GPU.

L	χ	single (s)	4-GPU (s)	speedup
50	64	11.1	31.7	$0.35\times$
50	128	190.3	62.8	$3.03\times$
50	256	311.0	123.7	$2.51\times$
100	64	35.7	80.8	$0.44\times$

954 The directional agreement with the analytical bound is encouraging, but
 955 the comparison is exploratory: the two harnesses were not run under matched
 956 warmup/polish or chi-schedule, the multi-GPU configurations are single-rep
 957 without crash-filter or Wilson CI, and the inter-device transfer path was
 958 not instrumented. A statistically-pinned $N=10$ multi-GPU campaign with
 959 matched harness is left for follow-up work, which will require multiple coordinated multi-MI300X windows beyond what was available for the present submission. We make no headline performance claim for multi-GPU `pdmrg` in this paper.

963 5.7.2. Riemannian-manifold optimizers (`rlbfgs-gpu`, `radam-gpu`)

964 DMRG sweeps the chain site by site, performing a local eigensolve at
 965 each bond. An alternative is to optimize the entire MPS as a single point
 966 on the matrix-product manifold using a Riemannian gradient method. Two
 967 such optimizers are implemented as polish-after-warmup steps that take a
 968 partially-converged MPS and run additional manifold-gradient updates: a
 969 Riemannian L-BFGS (`rlbfgs-gpu`, two-loop recursion + on-the-fly vector
 970 transport + Armijo line search + cautious-pair update) and a Riemannian
 971 Adam (`radam-gpu`). Both pass correctness tests across all challenge sizes
 972 (Heisenberg $L=50-200$, Josephson $L=16-64$, TFIM $L=50-200$) to the truncation floor.

974 *Analytical context.* A DMRG sweep performs L local Lanczos eigensolves
 975 with tight tolerance. Each local eigensolve is many implicit optimizer steps
 976 using exact second-order information from the Krylov subspace on a sub-
 977 problem that fits the MPS gauge exactly. A Riemannian gradient step is one
 978 first/second-order step on the full chain with worse conditioning from the
 979 tangent-space projection that maintains the manifold constraint. The work-

per- ΔE comparison therefore favours DMRG sweeps in the regimes where local eigensolves are well-conditioned, namely gapped 1D ground states with moderate χ . Where Riemannian optimization should help is precisely where DMRG sweeps are not natural or not cheap: long-range MPOs (per-bond cost rises faster than $\chi^3 d^2$), multi-layer tensor networks (no clean alternating-direction sweep gauge), excited-state targeting, or tangent-space time evolution.

Preliminary measurements (Heisenberg, MI300X). At $L=50$, $\chi=64$, after one DMRG1 warmup sweep ($E_0 = -21.972110254842$, $|g| = 2.5 \times 10^{-6}$): 100 Riemannian-LBFGS polish steps gave $\Delta E = -3.7 \times 10^{-10}$ in 166 s; 100 Riemannian-Adam steps gave $\Delta E = +2 \times 10^{-10}$ in 152 s with a transient regression of $+1.4 \times 10^{-9}$ at epoch 50; one additional DMRG1 sweep gave $\Delta E = -1.5 \times 10^{-8}$ in roughly $4\times$ less wall time. At $L=200$, $\chi=128$, the same pattern: 100 LBFGS steps $\Delta E = -3.3 \times 10^{-7}$ (166 s), 100 Adam steps $\Delta E = -1.7 \times 10^{-7}$ (152 s), one extra DMRG1 sweep $\Delta E = -1.6 \times 10^{-6}$ (38 s). Across the tested 1D ground-state envelope, one extra DMRG1 sweep delivers an order of magnitude more ΔE in $4\times$ less wall time than 100 manifold-gradient steps; the Riemannian polish does not earn its keep. Backing data: `benchmarks/data/` (radam/rbfgs result files, single-rep exploratory).

Future work. The Riemannian-optimizer prototypes are kept for the regimes where the analytical context above does favour them; we do not attempt those regimes in this paper.

5.7.3. Common posture

Both directions in this subsection are deliberately framed as exploratory: each has analytical reasoning that predicts the regime of competitiveness, preliminary measurements consistent with that prediction, and a clean future-work boundary. Reporting them here serves three purposes: (i) it documents that the directions were considered and not silently dropped; (ii) it gives a complexity-grounded answer to the natural reviewer question of why they are not in the headline tables; and (iii) it preserves the analytical bounds so that future work in the regimes where these methods are predicted to help can build on them rather than re-deriving.

1012 6. Discussion

1013 The results above document a series of independent optimization attempts—
1014 GPU GEMM acceleration, two parallelization strategies, three alternative
1015 algorithms—that all fail to deliver consistent speedups. Rather than ana-
1016 lyzing each failure in isolation, we now unify them under three structural
1017 explanations.

1018 6.1. The small-matrix wall

1019 At $\chi \leq 50$, three distinct overhead mechanisms conspire to make the
1020 GPU slower than a single CPU core.

1021 First, *GPU launch overhead*: each rocBLAS GEMM call incurs $\sim 5\text{--}10\ \mu\text{s}$
1022 of kernel dispatch latency, and a single Lanczos iteration requires hundreds
1023 of such calls. At $\chi = 50$, the arithmetic per GEMM (a 50×50 multiply
1024 taking $\sim 0.25\ \mu\text{s}$ at peak) is smaller than the launch overhead itself, so the
1025 GPU spends more time dispatching work than performing it.

1026 Second, *CPU threading overhead*: the same small matrices that starve the
1027 GPU would also penalize multi-threaded CPU BLAS if multiple threads were
1028 used. The CPU harness is hardcoded to a single thread (`OMP_NUM_THREADS=1`),
1029 so the 93% single-thread statistic reflects the harness design rather than a
1030 measured multi-thread comparison. At these sizes, a single core streaming
1031 through L2-resident data is the most efficient executor available.

1032 Third, *iterative algorithm overhead*: Block-Davidson increases the total
1033 number of matrix operations per bond (Newton–Schulz, as a CPU-prototype
1034 design, shares the same limitation). When each operation is already overhead-
1035 dominated, multiplying the operation count makes the situation strictly
1036 worse.

1037 These three effects are facets of the same underlying cause: at $\chi \leq 50$,
1038 DMRG’s per-bond linear algebra problems are too small to amortize any form
1039 of parallelism overhead, whether hardware (GPU wavefronts, CPU threads)
1040 or algorithmic (iterative refinement loops).

1041 6.2. The SVD ceiling

1042 At $\chi \geq 128$, the GPU’s GEMM throughput does overcome launch over-
1043 head for the Lanczos eigensolver—but this victory is pyrrhic. The profiling
1044 in Section 5.6 shows that SVD consumes 97–98% of per-sweep wall time
1045 at $\chi = 256$, imposing a ceiling that no eigensolver or environment-update
1046 optimization can breach.

The ceiling is structural. DMRG requires a full SVD (all singular values and vectors) at every bond to truncate the MPS, and LAPACK’s QR-iteration algorithm [24] performs this in a single $O(n^3)$ pass with low constant factors. Any iterative replacement must either match this efficiency—difficult, since QR-iteration exploits dense matrix structure that iterative methods do not—or accept a higher total FLOP count and rely on hardware throughput to compensate. Newton–Schulz attempts the latter strategy but requires up to 30 GEMM iterations, a $\sim 30\times$ FLOP increase, and at $n = \chi d \leq 512$ the matrices remain below the ~ 2048 threshold where the MI300X’s MFMA units reach peak occupancy.

GPU SVD via rocsolver follows the same QR-iteration algorithm as LAPACK but adds device overhead (kernel launches, memory management) that exceeds the compute savings at $\chi \leq 256$; the measured 13% improvement at $\chi = 256$ and the crossover estimate are discussed in the SVD path choice paragraph of Section 5.6.

6.3. The A2DMRG paradox

A2DMRG [19] illustrates a subtler failure mode: the algorithm is mathematically sound but operationally self-defeating. The warm-up experiment (Table 8) shows that each serial DMRG2 sweep buys 2–3 orders of magnitude in A2DMRG accuracy. By 5 warm-up sweeps the parallel phase converges in 1–2 iterations—but by then serial DMRG2 has already nearly converged, rendering the parallel phase redundant.

The underlying issue is that A2DMRG’s additive (Jacobi-like) correction cannot compete with standard DMRG’s multiplicative (Gauss–Seidel-like) sweep for sequential convergence rate. Parallelism could compensate if the per-iteration cost were sufficiently reduced, but A2DMRG’s linear combination phase inflates the bond dimension to $(L - 1) \times \chi$, and the subsequent TT-SVD compression—an $O(L\chi^3)$ serial SVD chain—is both slow and lossy. At $L = 32$, $\chi = 20$, the 31:1 compression ratio destroys the information that the parallel local solves produced.

This is not merely an implementation issue. The algorithm’s design requires that the compression ratio grow linearly with L , creating a fundamental tension between system size and accuracy. The original paper [19] operates in a regime ($d \leq 24$, rank-growth initialization) where this tension is manageable; our lattice-model tests expose the scaling limit. Within our tested regime A2DMRG was not competitive on wall-clock time; the regime

1083 where it was designed to win—large d , large χ , many parallel processors—lies
1084 above our bond-dimension ceiling.

1085 6.4. When would GPU DMRG win?

1086 The three structural barriers—small-matrix overhead, SVD ceiling, and
1087 compression loss—point to the conditions under which GPU DMRG would
1088 provide clear advantages:

1089 *Large bond dimensions* ($\chi \gtrsim 1024$).. Extrapolating sweep-time scaling from
1090 $\chi \in \{64, 128, 256, 512\}$, we *conjecture* a CPU/GPU crossover near $\chi \sim 1024$
1091 on the MI300X: both GEMM and SVD matrices would exceed the size thresh-
1092 olds where GPU hardware reaches high occupancy, and Newton–Schulz’s
1093 $30\times$ FLOP overhead could in principle be offset by a $> 30\times$ GPU/CPU
1094 throughput ratio. We have not measured this regime; the conjecture rests
1095 on (i) the empirical exponent fit and (ii) the asymptotic χ^3 GEMM/SVD
1096 argument, and should not be cited as a measured result. Ganahl et al. [?
1097] provide an independent data point on NVIDIA hardware at large χ using
1098 tensor processing units, confirming that a CPU/GPU crossover is real and
1099 architecture-portable. The numerical stability of Newton–Schulz at large
1100 condition numbers remains a separate open question.

1101 *Multi-GPU parallelism*.. The real-space parallel DMRG algorithm [13] was
1102 designed for distributed-memory systems with one segment per processor.
1103 On a single GPU, the segment parallelism competes for the same compute
1104 resources. With multiple GPUs, each segment would have dedicated compute
1105 capacity, and the Amdahl’s law bottleneck from serial coupling sweeps could
1106 be addressed through pipelining.

1107 *Alternative SVD approaches*.. Randomized SVD [26] provides large speedups
1108 ($5\text{--}6\times$ at $\chi = 128\text{--}256$) in two-site DMRG, where the per-bond matrix has d
1109 times more singular values than the truncation retains; the ablation results
1110 in Section 5.5 confirm this. The same flag is a no-op in single-site DMRG,
1111 where the matrix never has more singular values than $\chi_{\max}$. Mixed-precision
1112 approaches (single-precision for warmup, double for final convergence) could
1113 halve the SVD time for the majority of sweeps and remain unexplored in this
1114 study.

1115 *Launch-overhead amortization for parallel variants..* HIP graph capture of
 1116 `apply_heff` (Section 5.5) yields a $1.8\times$ speedup at $\chi = 256$ in `pdmrg-`
 1117 `gpu`, where the segment-parallel architecture launches many short-lived per-
 1118 segment kernels. For variants whose per-call work is already large enough to
 1119 amortize launch latency—`dmrg-gpu`, `dmrg2-gpu`—the same flag is flat. This
 1120 is the second instance of the broader pattern: micro-optimizations that tar-
 1121 get the measured local bottleneck of a specific variant pay off; those that
 1122 attempt to redistribute work without addressing the bottleneck do not.

1123 At $\chi \leq 256$ the arithmetic intensity of a single MPS update is too low
 1124 to amortize host-device transfer and kernel launch overhead in our default
 1125 Lanczos path; on this hardware (single MI300X, ROCm 7.2) and at this
 1126 bond dimension, an optimized single-thread CPU stack remains competitive.
 1127 We do not generalize beyond this regime: the ablation results in § 5.5 show
 1128 that algorithmic changes (RSVD, LANCZOS_GRAPH) already shift the
 1129 crossover for specific variants, and § 6.4 outlines why we expect $\chi \gtrsim 1024$
 1130 to favour the GPU. Whether smaller- χ GPU DMRG is *useful* depends on
 1131 workload (e.g. parameter sweeps, time evolution) considerations outside this
 1132 paper.

1133 6.5. Limitations and threats to validity

1134 We document here the principal threats to validity that a reader should
 1135 bear in mind when interpreting the benchmark data.

1136 *Hardware scope..* All GPU benchmarks were performed on a single AMD
 1137 Instinct MI300X (gfx942, ROCm 7.2). No A100, H100, or B200 data are
 1138 included; the CUDA port in `gpu-cuda/` is scaffolding only and has not been
 1139 compiled or run. No multi-GPU experiments were performed.

1140 *CPU baseline caveats..* The host CPU is an Intel Xeon Platinum 8470 (MI300X
 1141 system) or 8480C (H100 system), not AMD EPYC as an earlier draft stated.
 1142 The benchmark harness is hardcoded to `OMP_NUM_THREADS=1` (`run_mi300x_challenge.py:382`);
 1143 no multi-thread sweep was run. The OpenBLAS version and linkage path
 1144 are captured per-run by `ldd` but there is no build script in the repository,
 1145 and `LD_LIBRARY_PATH` is not set; OpenBLAS provenance cannot be fully ver-
 1146 ified from the published artefacts. Cotengra is listed as a dependency in the
 1147 Program Summary but is never imported by `quimb_runner.py`; the quimb
 1148 baseline uses `opt_einsum` greedy path. The quimb bond dimension schedule
 1149 is flat (`bond_dims=chi`); the idiomatic graduated schedule was not used.

1150 *SVD path..* All deterministic SVD calls use `dgesvd` (QR-iteration); `dgesdd`
 1151 (divide-and-conquer) was not tested. The Block-Davidson `-opt` variants
 1152 (`dmrg-gpu-opt`, `dmrg2-gpu-opt`) force a CPU host round-trip for SVD by
 1153 design; the primary `-gpu` variants and `pdmrg-gpu-opt` use on-device roc-
 1154 solver by default.

1155 *Algorithmic scope..* All GPU DMRG variants are strict single-site (no DMRG3S,
 1156 no noise injection, no subspace expansion [?]). Warmup and polish sweep
 1157 counts are capped at ≤ 2 per project convention.

1158 *Provenance gaps..* Only 15 of 116 result JSON files carry full provenance
 1159 fields (`git_sha`, `binary_sha256`, `cli_args`, `env_snapshot`). All GPU ven-
 1160 dor fields read `unknown` due to a `rocm-smi` parse failure at run time. Two
 1161 ablation JSON files contain `rc=-6` crash data that was counted as a valid
 1162 run result (later reruns superseded one; the other stands).

1163 6.5.1. Physics validation not performed

1164 The following standard convergence checks were not carried out and re-
 1165 main open:

- 1166 • **TFIM central charge.** At the critical point $h/J = 1$, the entangle-
 1167 ment entropy satisfies $S(L) = (c/6) \ln L + \text{const}$ with $c = 1/2$. Fitting
 1168 this scaling from the converged MPS bond entropy would confirm that
 1169 the simulation correctly captures the critical ground state.
- 1170 • **Heisenberg energy variance.** The quantity $\langle H^2 \rangle - \langle H \rangle^2$ per site
 1171 should be $< 10^{-6}$ at a converged bond dimension. This is computable
 1172 from existing MPO contractions and would quantify truncation error
 1173 more directly than the energy error threshold.
- 1174 • **Heisenberg E/L vs. Bethe ansatz.** The thermodynamic limit en-
 1175 ergy density is $e_0 = 1/4 - \ln 2 \approx -0.4431$ [21]. Reporting the fractional
 1176 deviation per (L, χ) cell from existing JSON data would contextualise
 1177 the accuracy of each implementation.

1178 These checks are deferred; results in this paper are validated only against
 1179 quimb reference energies to $|\Delta E| < 10^{-10}$.

7. Conclusions

We have presented a systematic benchmark of eleven DMRG implementations on the AMD MI300X GPU, spanning three physics models and a range of system sizes and bond dimensions. Our main findings are:

1. The GPU vs. CPU crossover depends on both χ and L . At $\chi = 128$, the GPU wins 100% of short-chain ($L \leq 20$) configurations but only 71% at $L = 32$ –64 and 50% at $L \geq 100$. At $\chi \leq 50$, the CPU wins all medium-to-large systems ($L \geq 32$).
2. When the CPU wins, 93% of CPU victories were measured at the single-thread setting that the harness hardcodes (`OMP_NUM_THREADS=1` throughout); we did not run a multi-thread sweep and cannot attribute this figure to BLAS contention. See limitations § 6.5. At $\chi \leq 50$, the matrices are small enough that thread synchronization overhead exceeds any parallel speedup, making a single core streaming through L2 cache the most efficient executor.
3. Two-site DMRG is $\sim 3\times$ faster than single-site at large systems ($L = 100$) due to faster convergence from adaptive bond dimension growth.
4. Replacing the single-vector Lanczos eigensolver with Block-Davidson ($b=4$) and routing SVD to a host-side LAPACK path uniformly fails to improve performance at $\chi \leq 256$ (0 wins out of 50 configurations). The joint BD+CPU-SVD confound prevents isolation of which factor dominates the slowdown; isolation is left to future work. Cross-segment batched GEMM is slower in 18 of 19 configurations.
5. A systematic ablation of six GPU micro-optimizations across all six `-gpu` and `-gpu-opt` variants finds two real wins and four flat outcomes (Section 5.5). Randomized SVD yields 5–6 $\times$ speedups in two-site DMRG, where the per-bond matrix has d times more singular values than the truncation retains. HIP graph capture of `apply_heff` yields 1.2–1.8 $\times$ in segment-parallel `pdmrg-gpu`, where launch latency is a meaningful fraction of per-iteration wallclock. Both wins target the locally-binding bottleneck of a specific variant; the same flags are flat where their target bottleneck does not bind. Device-side truncation, fused Lanczos kernels, sparse-MPO compaction, and MFMA-friendly bond-dim padding are all flat across variants.
6. CPU SVD consumes 97–98% of per-sweep wall time at $\chi = 256$. This bottleneck cannot be addressed by any optimization that targets the remaining 2–3% of runtime.

- 1217 7. Parallel DMRG on a single GPU is slower than serial two-site DMRG
1218 at all tested sizes, limited by the serial coupling sweep required to
1219 maintain global MPS consistency.
- 1220 8. The additive two-level parallel DMRG (A2DMRG) algorithm [19] achieves
1221 machine-precision accuracy for $L \leq 20$ but degrades to 10^{-5} errors at
1222 $L = 64$ due to TT-SVD compression loss. With $P = 2$ ranks, it is $2\text{--}4\times$
1223 slower than serial DMRG2, and sufficient serial warm-up renders the
1224 parallel phase redundant. This is the first independent reproduction of
1225 the algorithm and the first test on lattice spin models.

1226 *Practitioner bottom line..* For production DMRG at $\chi \leq 256$, use quimb (or
1227 equivalent) with single-threaded BLAS on CPU. Move to GPU single-site
1228 DMRG only at $\chi \geq 128$ on short chains ($L \leq 20$), and to GPU two-site
1229 DMRG when convergence is slow at large L . Do not invest in parallelism or
1230 iterative SVD replacements at these bond dimensions.

1231 These negative results delineate the boundary between problem regimes
1232 where GPU algorithmic variants can and cannot compete with mature CPU
1233 linear algebra. At moderate bond dimensions ($\chi \leq 256$), LAPACK’s opti-
1234 mized SVD and the Lanczos algorithm’s efficiency for single-eigenvalue prob-
1235 lems make them difficult targets. The regime where BLAS-3 replacements
1236 would be expected to help— $\chi \gtrsim 1024$ on multi-GPU systems—remains to be
1237 explored. More broadly, our experience suggests that porting an algorithm
1238 to a GPU is not merely a matter of mapping operations to hardware: the
1239 structure of the algorithm itself—how much serial factorization it requires,
1240 how small its inner matrices are, how lossy its compression steps become—
1241 determines whether the port can ever pay off.

1242 *Future directions..* DMRG3S subspace expansion on GPU: implementing the
1243 Hubig–McCulloch–Schollwöck–Wolf noise term [?] in HIP kernels would
1244 allow strict single-site to grow Schmidt sectors without switching to two-site,
1245 potentially closing the convergence-sweep gap and broadening the parameter
1246 window where single-site is competitive.

1247 All source code (11 implementations in Python, Python/MPI, and C++/HIP),
1248 benchmark scripts, and raw benchmark data are available at [https://github.](https://github.com/yue00/dmrg-implementations)
1249 [com/yue00/dmrg-implementations](https://github.com/yue00/dmrg-implementations).

1250 CRediT authorship contribution statement

1251 **Yaakoub El Khamra:** Conceptualization, Methodology, Software, Val-
1252 idation, Formal analysis, Investigation, Data curation, Writing – original
1253 draft, Writing – review & editing, Visualization.

1254 Declaration of competing interest

1255 The authors declare that they have no known competing financial inter-
1256 ests or personal relationships that could have appeared to influence the work
1257 reported in this paper.

1258 Data availability

1259 The source code, benchmark scripts, and raw benchmark data are avail-
1260 able at <https://github.com/yye00/dmrg-implementations>. The reposi-
1261 tory includes build instructions for ROCm 7.2 on AMD MI300X (gfx942).

1262 Benchmark results are stored in JSON format. The ablation study (Ta-
1263 ble 6) data reside under `benchmarks/data/gpu_ablation/<campaign-tag>/<variant>/results.`
1264 one file per variant per tagged campaign. Challenge-size results (Tables 2–5
1265 and Tables 8–12) are under `benchmarks/paper_results/mi300x/challenge/<campaign-tag>/`.
1266 The GPU-optimisation scaling study (Table 5) additionally uses `benchmarks/data/bench_opt_res`
1267 A per-table index with sha256 checksums will be provided in `MANIFEST.json`
1268 at the repository root upon resubmission.

1269 **Note on provenance.** The files `paper_results/mi300x/results.json`
1270 and `gpu_opt_bench.json` named in earlier drafts predate the provenance
1271 harness and do not embed `provenance.*` fields. Two ablation JSONs pro-
1272 duced on 2026-04-20/21 from git commit `c26f0bd` contained crash-contaminated
1273 medians and have been moved to `benchmarks/data/gpu_ablation/superseded/`
1274 with accompanying READMEs; they are not used for any table in this pa-
1275 per. Clean re-runs under a single campaign tag with the hardened harness
1276 (returncode filter, `data_quality` gate) will replace those cells before resub-
1277 mission.

1278 Acknowledgments

1279 We acknowledge access to AMD Instinct MI300X hardware through Hot
1280 Aisle, Inc. for the benchmarking reported in this work.

References

- [1] S. R. White, Density matrix formulation for quantum renormalization groups, *Physical Review Letters* 69 (1992) 2863–2866. doi:10.1103/PhysRevLett.69.2863.
- [2] S. R. White, Density-matrix algorithms for quantum renormalization groups, *Physical Review B* 48 (1993) 10345–10356. doi:10.1103/PhysRevB.48.10345.
- [3] U. Schollwöck, The density-matrix renormalization group in the age of matrix product states, *Annals of Physics* 326 (2011) 96–192. doi:10.1016/j.aop.2010.09.012.
- [4] R. Orús, A practical introduction to tensor networks: Matrix product states and projected entangled pair states, *Annals of Physics* 349 (2014) 117–158. doi:10.1016/j.aop.2014.06.013.
- [5] M. Fishman, S. R. White, E. M. Stoudenmire, The ITensor software library for tensor network calculations, *SciPost Physics Codebases* (2022) 4doi:10.21468/SciPostPhysCodeb.4.
- [6] J. Gray, quimb: A Python package for quantum information and many-body calculations, *Journal of Open Source Software* 3 (2018) 819. doi:10.21105/joss.00819.
- [7] C. Nemes, G. Barcza, Z. Nagy, Ö. Legeza, P. Szolgay, The density matrix renormalization group algorithm on kilo-processor architectures: Implementation and trade-offs, *Computer Physics Communications* 185 (2014) 1570–1581. doi:10.1016/j.cpc.2014.02.surface.
- [8] J. Eisert, M. Cramer, M. B. Plenio, Colloquium: Area laws for the entanglement entropy, *Reviews of Modern Physics* 82 (2010) 277–306. doi:10.1103/RevModPhys.82.277.
- [9] I. V. Oseledets, Tensor-train decomposition, *SIAM Journal on Scientific Computing* 33 (2011) 2295–2317. doi:10.1137/090752286.
- [10] S. R. White, Density matrix renormalization group algorithms with a single center site, *Physical Review B* 72 (2005) 180403. doi:10.1103/PhysRevB.72.180403.

- 1312 [11] Advanced Micro Devices, rocBLAS: ROCm basic linear algebra subpro-
1313 grams, <https://rocm.docs.amd.com/projects/rocBLAS/> (2024).
- 1314 [12] Advanced Micro Devices, ROCm documentation, <https://rocm.docs.amd.com/>, version 7.2 (2024).
1315
- 1316 [13] E. M. Stoudenmire, S. R. White, Real-space parallel density matrix
1317 renormalization group, *Physical Review B* 87 (2013) 155137. doi:10.
1318 1103/PhysRevB.87.155137.
- 1319 [14] G. Schulz, Iterative Berechnung der reziproken Matrix, *Zeitschrift für*
1320 *Angewandte Mathematik und Mechanik* 13 (1933) 57–59. doi:10.1002/
1321 zamm.19330130111.
- 1322 [15] Y. Nakamura, N. J. Higham, Optimized stable scaled Newton–Schulz
1323 iteration for the matrix sign function and polar decomposition, *SIAM*
1324 *Journal on Matrix Analysis and Applications* (2020).
- 1325 [16] E. R. Davidson, The iterative calculation of a few of the lowest eigen-
1326 values and corresponding eigenvectors of large real-symmetric matrices,
1327 *Journal of Computational Physics* 17 (1975) 87–94. doi:10.1016/0021-
1328 9991(75)90065-0.
- 1329 [17] B. Liu, The simultaneous expansion method for the iterative solution of
1330 several of the lowest eigenvalues and corresponding eigenvectors of large
1331 real-symmetric matrices, *Numerical Algorithms in Chemistry: Algebraic*
1332 *Methods* 49 (1978).
- 1333 [18] Y. Zhou, Y. Saad, M. L. Tiago, J. R. Chelikowsky, Self-consistent-
1334 field calculations using Chebyshev-filtered subspace iteration, *Journal*
1335 *of Computational Physics* 219 (2006) 172–184. doi:10.1016/j.jcp.
1336 2006.03.017.
- 1337 [19] L. Grigori, S. B. Hassan, An additive two-level parallel algorithm
1338 for the density matrix renormalization group method, *arXiv preprint-*
1339 *ArXiv:2505.23429v2* (2025). arXiv:2505.23429.
- 1340 [20] J. Gray, S. Kourtis, Hyper-optimized tensor network contraction, *Quan-*
1341 *tum* 5 (2021) 410. doi:10.22331/q-2021-03-15-410.

- [21] H. Bethe, Zur Theorie der Metalle. I. Eigenwerte und Eigenfunktionen der linearen Atomkette, Zeitschrift für Physik 71 (1931) 205–226. doi:10.1007/BF01341708.
- [22] R. Fazio, H. van der Zant, Quantum phase transitions and vortex dynamics in superconducting networks, Physics Reports 355 (2001) 235–334. doi:10.1016/S0370-1573(01)00022-9.
- [23] P. Pfeuty, The one-dimensional Ising model with a transverse field, Annals of Physics 57 (1970) 79–90. doi:10.1016/0003-4916(70)90270-8.
- [24] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, D. Sorensen, LAPACK Users’ Guide, 3rd Edition, SIAM, 1999.
- [25] C. Lanczos, An iteration method for the solution of the eigenvalue problem of linear differential and integral operators, Journal of Research of the National Bureau of Standards 45 (1950) 255–282.
- [26] N. Halko, P.-G. Martinsson, J. A. Tropp, Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions, SIAM Review 53 (2011) 217–288. doi:10.1137/090771806.

Appendix A. Complete benchmark results

Tables A.13–A.15 report wall times (seconds) for all converged benchmark configurations. For CPU implementations (Q-D1: quimb-dmrg1, Q-D2: quimb-dmrg2), the reported time is the single-threaded run (OMP_NUM_THREADS=1). For GPU implementations (D1-GPU: dmrg-gpu, D2-GPU: dmrg2-gpu, PD-GPU: pdmrg-gpu with $P = 2$ segments), all runs use a single MI300X with CPU SVD. The fastest implementation for each configuration is shown in **bold**. A dash (—) indicates the implementation was not tested or did not converge for that configuration.

The full results reinforce the narrative developed in Sections 5–6:

- **GPU single-site (D1-GPU) dominates the TFIM:** it wins 10 of 13 TFIM configurations, including all of $L \leq 20$. The TFIM’s small MPO bond dimension ($D = 3$) and fast convergence (2–3 sweeps) keep per-bond costs low, favoring the GPU even at moderate χ —precisely

Table A.13: Complete wall times (seconds) for the Heisenberg spin- $\frac{1}{2}$ chain (OBC, $d = 2$, $D = 5$, real). Bold indicates fastest.

L	χ	Q-D1	Q-D2	D1-GPU	D2-GPU	PD-GPU
12	20	0.81	1.10	0.67	0.52	0.71
12	50	0.78	1.06	0.42	0.56	0.65
12	128	0.83	0.99	0.42	0.54	0.66
20	20	1.11	1.12	2.31	1.07	1.89
20	50	1.08	1.43	1.42	1.45	2.26
20	128	2.05	4.82	1.36	2.36	3.36
32	20	1.50	1.69	5.06	2.27	—
32	50	1.82	2.91	6.20	2.91	—
32	128	5.24	15.24	4.24	5.67	—
64	50	5.67	5.05	26.99	9.27	24.50
64	128	20.59	37.69	24.01	22.80	—
100	50	11.36	8.50	74.22	43.09	—
100	128	40.70	75.68	149.40	53.57	—

the small-matrix regime where low per-bond overhead matters most (Section 6.1).

- **Josephson is hardest for the GPU:** complex arithmetic doubles the GEMM cost, and $d = 5$ inflates the two-site eigensolve dimension to $\chi d^2 = 25\chi$. D2-GPU is never the fastest for Josephson at $\chi \geq 50$. The CPU (Q-D1) wins all Josephson configurations at $\chi = 50$, and the GPU only overtakes at $\chi = 128$ for $L \leq 32$ —consistent with the SVD ceiling becoming the binding constraint once the eigensolver is fast enough (Section 6.2).

- **Parallel DMRG (PD-GPU) never wins:** across all 28 configurations where it was tested, pdmrg-gpu is always slower than both the best serial GPU and best CPU implementation. Its closest approach to competitiveness is Heisenberg $L = 12$, $\chi = 20$ (0.71 s vs. 0.52 s for D2-GPU), but even there it loses. Every parallel segment sweep still pays the full per-bond SVD cost, and the serial coupling phases add to that baseline.

- **Large Heisenberg systems favor CPU or two-site GPU:** at $L \geq$

Table A.14: Complete wall times (seconds) for the Josephson junction array (OBC, $d = 5$, $D = 4$, complex). Bold indicates fastest.

L	χ	Q-D1	Q-D2	D1-GPU	D2-GPU	PD-GPU
8	20	1.46	1.31	2.67	1.49	4.70
8	50	1.25	3.18	1.10	1.16	19.05
8	128	3.55	6.52	0.97	2.54	20.01
16	20	2.30	3.53	6.19	6.38	12.40
16	50	3.73	16.32	10.70	8.03	55.61
16	128	26.19	228.10	4.67	29.17	314.34
32	50	17.07	59.69	32.30	126.23	239.93
32	128	107.98	891.90	33.02	107.87	800.42
48	50	31.00	360.51	64.69	244.94	326.85
48	128	172.77	—	191.16	1347.92	—
64	50	41.62	1287.38	88.55	568.37	—
64	128	300.79	—	295.82	981.48	—

64, the CPU (Q-D1 or Q-D2) wins at $\chi \leq 128$. When the GPU wins (e.g., $L = 32$, $\chi = 128$), it is always D1-GPU. D2-GPU is competitive at $L = 64$, $\chi = 128$ (22.80 s vs. 24.01 s for D1-GPU) but does not win—the two-site SVD cost ($d^2\chi$ matrices) pushes it further into the SVD-dominated regime.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used Anthropic Claude (models Opus 4.6/4.7) for (a) drafting and revising portions of the manuscript prose, including the rewrite captured in commit `40af7bf` and the CPC submission preparation (`bb8bcd0`); and (b) generating, refactoring, and reviewing source code in the `gpu-rocm/` and `cpu/` trees. Approximately 38% of the 113 git commits in the public repository (43 commits) were authored by an AI assistant operating under human direction; all such commits were reviewed and accepted by the human author(s) before merge. After using these tools, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

Table A.15: Complete wall times (seconds) for the transverse-field Ising model at criticality (OBC, $d = 2$, $D = 3$, real). Bold indicates fastest.

L	χ	Q-D1	Q-D2	D1-GPU	D2-GPU	PD-GPU
12	20	0.77	0.79	0.39	0.51	5.18
12	50	0.78	0.80	0.41	0.49	4.54
12	128	0.83	0.84	0.41	0.52	1.38
20	20	0.88	1.00	0.69	1.01	12.41
20	50	1.03	1.20	0.98	1.76	16.46
20	128	1.95	1.87	1.52	3.47	13.48
32	20	1.14	1.26	1.50	1.50	18.71
32	50	1.64	1.83	1.78	3.78	29.75
32	128	5.81	6.17	4.89	13.46	45.78
64	50	3.97	4.80	4.96	8.67	27.36
64	128	22.11	26.59	20.45	51.75	45.55
100	50	7.66	9.31	8.36	15.97	59.89
100	128	40.81	55.31	36.31	96.98	—

Acknowledgements

The author(s) thank Anthropic Claude for code-generation and editorial assistance.